50

Technical Implementations From Real AI Projects

Production Patterns · Streaming · Agents · Automation · Blockchain · Security

- PART I — LLM Streaming & Agent APIs
- PART II — Multi-Agent Orchestration
- PART III — Playwright & Browser Automation
- PART IV — Real-Time Streaming & WebSockets
- PART V — Financial Data Systems
- PART VI — DeFi & Blockchain
- PART VII — Content & Media Pipelines
- PART VIII— Security & Identity
- PART IX — Database & Persistence Patterns
- PART X — Knowledge Extraction & RAG
- PART XI — Testing, Deployment & DevOps

github.com/rohith547 · Built with real production code

Table of Contents

Part 1 — LLM Streaming & Agent APIs

01. Streaming AI Agent API
02. Multi-Provider AI Factory
03. Zod-Validated AI Orchestrator
04. Real-Time LLM Streaming with ReadableStream

Part 2 — Multi-Agent Orchestration

05. Personal Multi-Agent System
06. Context-Tiered Identity Access Control
07. Call Memory Learning System
08. Fraud Detection in AI Voice Calls
09. CommandCore - 20-Service Telegram Hub
10. BullMQ Job Queue System

Part 3 — Playwright & Browser Automation

11. AgentBrowser - Async Playwright Wrapper for AI Agents
12. Playwright Autotrader Scraper
13. Bank Portal Automation with Session Persistence
14. Playwright Scheduling Agent

Part 4 — Real-Time Streaming & WebSockets

15. Binance WebSocket Price Feed with Reconnection
16. Multi-Signal Ensemble Trading System
17. FastAPI WebSocket Conversation Server
18. Server-Sent Events (SSE) Streaming in Next.js

Part 5 — Financial Data Systems

19. FRED API Macro Indicators
20. Weighted Market Score Engine
21. Kelly Criterion and Expected Value Engine
22. News Sentiment LLM Classifier
23. Plaid Banking Integration
24. Affordability Assessment Engine
25. Bill Anomaly Detection

Part 6 — DeFi & Blockchain

26. EIP712 Order Signing for Polymarket CLOB
27. Web3 Wallet Management on Polygon
28. Flash Loan Arbitrage Detection

Part 7 — Content & Media Pipelines

29. ElevenLabs TTS with Character Voice Mapping
30. Frame-by-Frame Video Animation Engine
31. AI Script Generation with Groq
32. Content Deduplication Brain
33. YouTube Data API v3 - Automated Video Upload
34. Telegram Review Gate
35. TF-IDF News Story Clustering

Part 8 — Security & Identity

36. Fernet Field-Level Encryption Vault
37. HMAC Constant-Time Comparison and SHA256 Hashing
38. Google OAuth2 Flow
39. FastAPI Lifespan Pattern

Part 9 — Database & Persistence Patterns

40. Prisma + TypeScript - Transaction-Safe Knowledge Pipeline
41. Async SQLite with aiosqlite
42. Alembic Migrations
43. Prisma Multi-Service ORM

Part 10 — Knowledge Extraction & RAG Patterns

44. YouTube Transcript + AI Knowledge Extraction Pipeline
45. Multi-Provider RAG - Gemini + Ollama Abstraction
46. Article Scraper - Summarize, Categorize, Store

Part 11 — Testing, Deployment & DevOps

47. Vitest Test Suite - Unit, Integration, and Bot Tests
48. Docker + Docker Compose - Production Containerization
49. IP-Based Rate Limiting with Supabase
50. Stripe Webhook + Checkout - Monetization Pattern

Preface

This book is not a collection of theoretical exercises. Every implementation documented in these 50 chapters was extracted from real, working AI projects built throughout 2025 and 2026. The code runs in production. The patterns solve real problems. The hard-won lessons come from actual debugging sessions, failed deployments, and late-night API error logs.

The projects range from a personal multi-agent system that manages scheduling, insurance, and travel, to a fully automated video content pipeline that generates, voices, animates, and publishes financial explainer videos to YouTube without human intervention. They include a DeFi arbitrage scanner, a bank portal automation system, a Telegram-based personal operating system with 20 concurrent services, and a knowledge vault that transforms YouTube lectures into queryable structured knowledge.

What unifies these projects is the engineering discipline required to take AI from demo to production. Streaming APIs that handle disconnections gracefully. Validation layers that catch LLM hallucinations before they corrupt databases. Rate limiting that prevents $5,000 cloud bills from API abuse. Encryption that protects sensitive data even when the database is compromised. These are the implementations that matter.

Each chapter follows a consistent structure: an Overview that explains why the implementation exists and what problem it solves, a How It Works section with specific technical details, actual code from the project repositories, Key Insight callout boxes distilling the most important learnings, and a Why It Matters section that situates the implementation in the broader landscape of production AI engineering.

The 50 implementations span 11 domains: LLM APIs, multi-agent orchestration, browser automation, real-time streaming, financial data systems, DeFi, content pipelines, security, database patterns, knowledge extraction, and deployment. Whether you are building your first AI application or your twentieth, you will find patterns here that apply directly to production problems you face today.

Build real things. Ship them. Learn from what breaks. These implementations are the evidence that AI engineering, done with production discipline, delivers systems that are actually useful.

Disclaimer

EDUCATIONAL PURPOSE ONLY: This book documents technical implementations from personal software engineering research projects. All code, algorithms, and system designs are presented strictly for educational and learning purposes.

NOT FINANCIAL ADVICE: Chapters covering financial topics — including Kelly Criterion position sizing, prediction market algorithms, DeFi arbitrage detection, and Plaid banking integration — are engineering case studies only. Nothing in this book constitutes financial advice, investment advice, or a recommendation to trade any financial instrument, cryptocurrency, or prediction market contract. Always consult a licensed financial advisor before making investment decisions.

NOT LEGAL ADVICE: This book does not constitute legal advice of any kind. Readers are responsible for ensuring that their use of any technology, API, or pattern described herein complies with applicable laws and regulations in their jurisdiction.

THIRD-PARTY SERVICES: References to third-party services (Polymarket, Binance, Plaid, Stripe, ElevenLabs, etc.) are for educational illustration only. The author has no affiliation with these companies. API terms of service, availability, and pricing may change.

USE AT YOUR OWN RISK: The implementations in this book are from personal research projects. They are not production-hardened, audited, or warranted for any particular purpose. The author makes no guarantees about correctness, security, or fitness for use.

COPYRIGHT: All original content in this book is copyright the author. Code examples may be used for personal learning and non-commercial educational purposes with attribution.

PART 1

LLM Streaming & Agent APIs

Chapter 1

Streaming AI Agent API

Next.js + Anthropic Claude

Project: agent-base

Overview

LLM streaming fundamentally reshapes the user experience of AI-powered applications — and user trust depends on it more than most engineers realize. In a traditional request-response cycle, the client sends a query, a spinner appears, and the user waits up to 15 seconds staring at nothing before a single byte arrives. Research on perceived performance consistently shows that users who see text appearing within 300ms form a positive impression of the system, while users who wait 5 or more seconds for a first byte begin to question whether the request even registered. Streaming eliminates this cold-start problem: the first tokens appear in the browser within 200-400 milliseconds. The model may still need 8 more seconds to finish generating, but the user is already reading and engaged. This is the literal difference between a product that feels alive and one that feels broken — and it costs nothing extra to implement.

The agent-base project implements a production-grade streaming API using Next.js App Router route handlers paired with Anthropic Claude Haiku. A critical mechanism is ReadableStream backpressure: when controller.enqueue() adds data faster than the browser can consume it, the stream's internal queue grows and the underlying transport signals the producer to slow down. This automatic flow control prevents memory exhaustion in the server process — model output never fully buffers in server RAM, it flows through the controller to the HTTP response body in real-time. If the stream errors mid-response (Claude returns a 500, the network drops, or the rate limit is hit partway through), the finally block must call controller.close() to terminate the response cleanly. The client receives a truncated but valid stream and the UI can display a partial response with an error indicator, rather than leaving the user with a hung connection and no feedback.

How It Works

- The POST route handler extracts the client IP from x-forwarded-for headers — the real IP behind any reverse proxy
- A Redis or Supabase lookup checks the IP's daily usage count before processing — blocking abuse before any model call
- A ReadableStream controller bridges the async Anthropic streaming API to the HTTP response body — no intermediate buffer
- anthropic.messages.stream() returns an async iterable of Server-Sent Events; we only care about content_block_delta

- Each text_delta event contains one partial chunk — typically 1-4 tokens — forwarded immediately to the browser
- TextEncoder converts UTF-8 string chunks to Uint8Array bytes because browser ReadableStream works with binary data
- The Content-Type: text/plain header tells the browser to render chunks progressively as they arrive via fetch()

app/api/agent/route.ts - Streaming route with IP rate limiting

```
export async function POST(req: NextRequest) {
  const { input } = await req.json()
  const ip = req.headers.get("x-forwarded-for") ?? "anonymous"
  const used = await getUsageCount(ip)
  if (used >= DAILY_LIMIT) {
    return NextResponse.json({ error: "limit_reached" }, { status: 429 })
  }
  await incrementUsage(ip)
  const encoder = new TextEncoder()
  const stream = new ReadableStream({
    async start(controller) {
      await streamAgentResponse(SYSTEM_PROMPT, input, (chunk) => {
        controller.enqueue(encoder.encode(chunk))
      })
      controller.close()
    },
  })
  return new Response(stream, {
    headers: { "Content-Type": "text/plain; charset=utf-8" },
  })
}
```

lib/claude.ts - Anthropic streaming wrapper with event filtering

```
export async function streamAgentResponse(
  system: string, message: string,
  onChunk: (text: string) => void
) {
  const stream = anthropic.messages.stream({
    model: "claude-haiku-4-5", max_tokens: 600,
    system, messages: [{ role: "user", content: message }],
  })
  for await (const event of stream) {
    if (event.type === "content_block_delta" &&
        event.delta.type === "text_delta") {
      onChunk(event.delta.text)
    }
  }
}
```

■ Why ReadableStream Over Server-Sent Events

ReadableStream works natively with fetch(), requires no special client-side EventSource setup, and supports backpressure. The browser reads response body chunks as they arrive, enabling progressive rendering without WebSocket infrastructure.

■ Streaming Cuts Perceived Latency by 70-85%

A 600-token response that takes 9 seconds to generate fully shows its first tokens in under 300ms. This difference is the line between users staying engaged and users leaving. Always stream when possible.

■■ Watch Out: Error Handling Mid-Stream

If Claude returns an error after streaming 50 tokens, the browser has already rendered those tokens — you cannot un-send them. Always wrap your streaming logic in try/catch and call controller.error() on failure — this signals the client ReadableStream reader that an error occurred, allowing the UI to display a partial response with an error indicator rather than silently stopping. Log the partial token count for debugging. On the client side, distinguish AbortError (user cancelled) from network errors (show retry button) from model errors (show 'something went wrong').

Why It Matters

Streaming is no longer a nice-to-have feature — it is the standard users expect from every AI-powered interface, and the difference between streaming and non-streaming is one of the largest UX improvements you can make to an existing AI product at essentially zero cost. This implementation handles the complete pipeline: rate-limited request ingestion, asynchronous model streaming, and byte-level forwarding to the browser. The clean separation between route handler, rate limiter, and Claude wrapper makes each component independently testable and replaceable. Every AI application you build — coding assistants, document analysis, chat interfaces — will use this pattern or a close variant of it.

Chapter 2

Multi-Provider AI Factory

OpenAI / Anthropic / Ollama / Mock

Project: commandcore

Overview

Production AI systems cannot afford to be locked into a single model provider. API outages, rate limit changes, pricing increases, and capability gaps all demand the ability to swap providers without touching application code. The Multi-Provider AI Factory in CommandCore solves this with a classic factory pattern: a single getAIProvider() function reads an environment variable and returns a provider-agnostic AIProvider interface.

This architecture supports four backends: OpenAI GPT models for general-purpose tasks, Anthropic Claude for reasoning, a local Ollama server for offline inference and zero-cost development, and a MockAIProvider that returns deterministic responses for testing. Every provider implements the same AIProvider interface, so the application layer never knows which backend is active.

How It Works

- A shared AIProvider TypeScript interface defines the contract: generate(), stream(), and embed() methods
- The factory reads AI_PROVIDER from environment config or process.env at startup
- Each concrete provider class wraps the official SDK behind the shared interface
- MockAIProvider returns configurable fixture responses for fully deterministic unit tests
- OllamaProvider hits a local HTTP server on port 11434, enabling zero-cost development
- Switching providers requires changing only a single environment variable

src/ai/provider.ts - Factory with four provider backends

```
export function getAIProvider(config?: AppConfig): AIProvider {
  const provider = config?.AI_PROVIDER
    ?? process.env.AI_PROVIDER ?? "mock"
  switch (provider) {
    case "ollama":    return new OllamaProvider(OLLAMA_URL, OLLAMA_MODEL)
    case "openai":    return new OpenAIProvider(process.env.OPENAI_API_KEY!)
    case "anthropic": return new AnthropicProvider(process.env.ANTHROPIC_API_KEY!)
    case "mock":      return new MockAIProvider()
    default:          throw new Error("Unknown provider: " + provider)
  }
}
```

src/ai/interface.ts - AIProvider contract with Mock implementation

```
export interface AIProvider {
  generate(prompt: string): Promise<string>
  stream(prompt: string, onChunk: (t: string) => void): Promise<void>
  embed(text: string): Promise<number[]>
}
export class MockAIProvider implements AIProvider {
  async generate(prompt: string) {
    return "[Mock] " + prompt.slice(0, 40)
  }
  async stream(_: string, onChunk: (t: string) => void) {
    for (const w of ["Mock", "streamed", "response"])
      onChunk(w + " ")
  }
  async embed(_: string) { return Array(1536).fill(0.1) }
}
```

■■ Vendor Lock-In Is a Real Risk

Anthropic deprecated claude-v1 with short notice. OpenAI retired text-davinci-003 within months of releasing gpt-3.5-turbo. Any system that hard-codes SDK calls throughout its codebase faces hours of refactoring each time. The factory pattern limits the blast radius to a single file.

■ Use Mock in CI/CD

Setting AI_PROVIDER=mock in GitHub Actions eliminates test flakiness from API rate limits and network failures. Tests run in milliseconds at zero API cost.

■■ Common Pitfall: Hard-Coding Model Names Across the Codebase

The factory pattern fails if model names like 'claude-haiku-4-5' or 'gpt-4o' appear in 30 different files. When Anthropic deprecates a model version, you face 30 find-and-replace operations. Route all model selection through the factory and store model identifiers in a single configuration constant. The factory should be the only file in your codebase that ever imports an AI SDK directly.

Why It Matters

The ability to swap AI providers at runtime is a fundamental requirement for production AI systems. CommandCore uses this factory to serve 20 different services from a single codebase, switching between Ollama for development and Anthropic for production. The pattern costs almost nothing to implement and pays dividends every time a provider has an outage or updates their API.

Chapter 3

Zod-Validated AI Orchestrator

Hallucination Guards and Schema Enforcement

Project: bills-tracker-v2

Overview

Language models are probabilistic systems that generate plausible text, not guaranteed structured data. When you ask a model to extract a bill amount, due date, and category from a bank statement, it might return a number where you expect a string, omit required fields, or invent a category that does not exist in your schema. The Zod-Validated AI Orchestrator treats LLM output as untrusted external input and validates it through strict runtime schemas before any data reaches the database.

This pattern wraps every LLM call in a validation-retry loop: the model is prompted to return JSON matching a specific Zod schema, the output is parsed and validated, and if validation fails the model is called again with the error message as correction context. After three failures, the orchestrator returns null rather than corrupting the database with bad data. This transforms AI-powered data pipelines from fragile experiments into production-safe systems.

How It Works

- Zod schemas define the exact shape, types, and allowed values of the expected LLM output
- The system prompt instructs the model to return only valid JSON matching the schema
- The raw LLM response is regex-matched to extract JSON then parsed with JSON.parse()
- schema.safeParse() validates the parsed object and returns typed result or error message
- On validation failure, the error message is fed back to the model as a correction prompt
- A retry loop with configurable max attempts prevents infinite loops on persistently bad outputs
- Nullable fields allow partial extraction - better to save partial data than fail completely

src/schemas/bill.ts - Zod schema with nullable fields for partial extraction

```
import { z } from "zod"

export const ExtractedBillSchema = z.object({
  name:      z.string().nullable(),
  amount:    z.number().nullable(),
  dueDate:   z.string().nullable(),
  cycleType: z.enum(["monthly", "annual", "weekly", "one-time"]).nullable(),
  category:  z.string().nullable(),
  merchant:  z.string().nullable(),
})
export type ExtractedBill = z.infer<typeof ExtractedBillSchema>
```

src/ai/extractor.ts - Validation-retry loop with error feedback

```
export async function extractBillWithRetry(
  rawText: string, maxAttempts = 3
): Promise<ExtractedBill | null> {
  let lastError = ""
  for (let i = 0; i < maxAttempts; i++) {
    const note = lastError ? `Fix: ${lastError}` : ""
    const raw = await ai.generate(SYSTEM + note + rawText)
    try {
      const json = JSON.parse(raw.match(/{[\s\S]*}/)?.[0] ?? raw)
      const result = ExtractedBillSchema.safeParse(json)
      if (result.success) return result.data
      lastError = result.error.message
    } catch (e) { lastError = String(e) }
  }
  return null
}
```

■■ Never Trust Raw LLM Output

LLMs can return strings for numeric fields, use different key names, or include extra prose outside the JSON block. Always validate. A bill amount of null is far safer than the string "seventeen dollars" entering your accounting system as NaN.

■ Combine Structured Output APIs with Zod

Anthropic and OpenAI both offer JSON mode APIs that constrain format at the model level. Use these for the first layer of defense, then add Zod validation as the second layer - the model produces JSON, your code validates it matches your exact schema.

■■ Common Pitfall: JSON Embedded in Prose Breaks Parsing

LLMs often return valid JSON wrapped in prose: 'Here is the extracted data: {"amount": 45.00}. Let me know if this looks correct!' The regex match for JSON blocks extracts the object, but if the prose itself contains curly braces (common in code examples), the regex grabs the wrong block. Use Anthropic or OpenAI structured output mode to force clean JSON responses at the model level, then validate with Zod as a second safety net.

Why It Matters

Hallucination guards are what separate AI prototypes from production systems. Bills-tracker-v2 processes real financial data where wrong amounts could cause missed payments. The validation-retry loop costs one or two extra API calls when it fires, but prevents corrupted data that would cost far more to debug. This pattern

applies to any system where LLM output feeds a database, a downstream service, or a financial calculation.

Chapter 4

Real-Time LLM Streaming with ReadableStream

Browser-Side Progressive Rendering

Project: agent-base

Overview

While Chapter 1 covered server-side streaming, this chapter dives into the client-side counterpart: reading a streaming HTTP response in the browser using the Fetch API and ReadableStream reader interface. The challenge is rendering chunks progressively in the UI while handling partial UTF-8 sequences, error recovery, and stream cancellation cleanly.

The agent-base project implements a complete client-side streaming hook that reads from a streaming fetch response, decodes chunks with a TextDecoder handling split multibyte characters, and progressively appends text to React state. The result is smooth, word-by-word text rendering that matches the ChatGPT and Claude.ai experience users expect from modern AI applications.

How It Works

- fetch() with method POST opens a persistent HTTP connection to the streaming API route
- response.body is a ReadableStream; getReader() returns a reader that pulls chunks on demand
- TextDecoder with stream:true correctly handles UTF-8 sequences split across chunk boundaries
- The reading loop calls reader.read() in a while(true) loop and breaks when done is true
- Each decoded chunk is appended to accumulated text triggering a React state update
- AbortController allows mid-stream cancellation closing the connection and releasing resources
- Error handling distinguishes user-initiated cancellation (AbortError) from real network failures

hooks/useAgentStream.ts - Client streaming hook with abort support

```
export function useAgentStream() {
  const [text, setText] = useState("")
  const [loading, setLoading] = useState(false)
  const abortRef = useRef<AbortController | null>(null)

  const stream = useCallback(async (input: string) => {
    abortRef.current = new AbortController()
    setText(""); setLoading(true)
    try {
      const res = await fetch("/api/agent", {
        method: "POST",
        body: JSON.stringify({ input }),
        signal: abortRef.current.signal,
      })
      const reader = res.body!.getReader()
      const decoder = new TextDecoder()
      while (true) {
        const { done, value } = await reader.read()
        if (done) break
        setText(t => t + decoder.decode(value, { stream: true }))
      }
    } catch (e) {
      if ((e as Error).name !== "AbortError")
        setText("Error: " + (e as Error).message)
    } finally { setLoading(false) }
  }, [])

  return { text, loading, stream, stop: () => abortRef.current?.abort() }
}
```

■ TextDecoder stream:true Is Critical

Without stream:true, TextDecoder treats each chunk as a complete UTF-8 sequence. This causes mangled characters when a multibyte emoji or accented character is split across two chunks. The stream option buffers incomplete sequences and flushes them with the next chunk.

■ Throttle UI Updates for Performance

If the model streams at 50 tokens per second and each triggers a React render, you get 50 renders per second. Use requestAnimationFrame or a 16ms debounce to batch updates and group several tokens per render for smooth display.

■■ Common Pitfall: Forgetting stream:true on TextDecoder

If you create TextDecoder without the {stream: true} option and a streaming chunk ends mid-UTF-8 sequence (common with emoji and special characters), the decoder outputs a replacement character instead of buffering the incomplete bytes. The next chunk starts with the remaining bytes of the incomplete sequence, producing garbled output. Always initialize with new TextDecoder('utf-8', {stream: true}) for streaming use cases.

Why It Matters

Progressive rendering is the defining UX characteristic of modern AI interfaces. Users who see text appearing instantly perceive the system as faster and more capable than one that shows a spinner for the same duration. This hook is fully reusable across React components. The abort controller integration ensures clean cancellation, preventing memory leaks and zombie requests when users navigate away mid-stream.

PART 2

Multi-Agent Orchestration

Chapter 5

Personal Multi-Agent System

Travel, Scheduling, Insurance and More

Project: personal-agents

Overview

A personal AI agent that handles only one type of task is a specialized tool. A personal AI system that coordinates multiple specialized agents is a force multiplier. The central engineering challenge is routing: when a user says "Book me a flight to NYC and add it to my calendar," how does the system know to invoke TravelAgent first, then hand the result to CalendarAgent? The personal-agents project solves this with an LLM-powered router that reads each message, compares it against concise capability declarations from all available agents, and names the single best-fit agent — all in a 200-400ms LLM call before any actual work begins. The latency cost of this routing call is small: a fast model like Claude Haiku costs fractions of a cent and returns in under half a second. This overhead is invisible to the user in a streaming interface, and it pays for itself by ensuring the right specialized agent handles each request.

The orchestration layer is a FastAPI server with WebSocket support for real-time conversation. Each agent is a Python class with a defined capability declaration, a context builder, and an execute method. The router uses an LLM call to classify requests and select the appropriate agent, then injects relevant user context including memory, profile, and history before execution begins. The flow is: user message arrives over WebSocket → RouterAgent sends a classification prompt to the LLM with all agent capability declarations → LLM returns one agent name → selected agent receives a rich context packet → agent executes its task using sub-tools → streaming response flows back to the user. If the router returns an agent name that does not match any registered agent, the fallback general agent catches the request and responds gracefully while logging the misclassification for capability refinement.

How It Works

- A FastAPI WebSocket endpoint maintains persistent connections — avoiding the 50-200ms HTTP handshake overhead on every conversation turn
- The RouterAgent classifies each message against agent capability declarations via LLM — declarations must be explicit include/exclude contracts, not vague descriptions
- Selected agents receive a context packet with user profile, memory, and history — so TravelAgent already knows your preferred airlines without you specifying them
- Agents invoke sub-tools like browser automation, external APIs, and database lookups — each awaited asynchronously so the pipeline never blocks

- Results flow back through the WebSocket as streaming text — each token forwarded as it arrives rather than buffered to completion
- A memory system stores per-agent interaction history — CalendarAgent learns you prefer 9am meetings after your first scheduling request
- Misclassified messages fall back to a general conversational agent — every fallback is logged to identify which capability declarations need refinement

agents/router.py - LLM-powered agent dispatcher with fallback

```
class AgentRouter:
    def __init__(self, agents: list, llm):
        self.agents = agents
        self.cap_list = "\n".join(
            f"{a.name}: {a.description}" for a in agents
        )

    async def route(self, message: str, ctx) -> str:
        prompt = (
            f"User request: {message}\n\n"
            f"Available agents:\n{self.cap_list}\n\n"
            "Reply with exactly one agent name."
        )
        agent_name = (await self.llm.generate(prompt)).strip()
        agent = next(
            (a for a in self.agents if a.name == agent_name),
            self.agents[-1]  # fallback: general agent
        )
        return await agent.execute(message, ctx)
```

agents/base.py - BaseAgent abstract class and AgentContext dataclass

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass
class AgentContext:
    user_id: str
    memory: dict
    history: list
    profile: dict

class BaseAgent(ABC):
    name: str
    description: str

    @abstractmethod
    async def execute(self, message: str, ctx: AgentContext) -> str:
        pass

class TravelAgent(BaseAgent):
    name = "travel"
    description = "flight searches, hotel booking, travel itineraries"

    async def execute(self, message: str, ctx: AgentContext) -> str:
        browser = AgentBrowser()
        results = await browser.search_flights(message, ctx.profile)
        return self.format_results(results)
```

■ Capability Declarations Are the Routing Engine

The quality of multi-agent routing depends entirely on how clearly each agent describes its capabilities. Vague descriptions lead to wrong routing. Write capability declarations as action-oriented sentences specifying exactly what the agent handles and what it does NOT handle.

■ Always Define a Fallback Agent

When the router misclassifies a message - and it will occasionally - the fallback general agent catches it and provides a reasonable response rather than an error. Log every fallback invocation to identify which capability declarations need refining.

■■

Common Pitfall: Vague Capability Declarations Break Routing

If TravelAgent's description is 'handles travel stuff' and CalendarAgent says 'manages schedule things,' the router will make arbitrary routing decisions. Write capability declarations as explicit include/exclude contracts: 'flight searches, hotel booking, itinerary creation — NOT calendar events or general knowledge questions.' Test routing accuracy by sending 30 representative messages and verifying which agent was selected each time. Aim for 95% routing accuracy before production deployment.

Why It Matters

Multi-agent systems are the architecture behind production AI assistants that handle diverse user requests with real capability. Rather than building one monolithic agent that knows everything poorly, this pattern lets you build specialized agents that know their domain deeply — TravelAgent that understands fare classes and booking windows, InsuranceAgent that knows your policy numbers and claim history. The routing layer adds minimal latency while dramatically improving response quality. As new capabilities are needed, you add a new agent class and update its capability declaration — the router adapts automatically without any changes to the dispatch logic. This is the pattern that scales from a personal assistant to an enterprise AI platform.

Chapter 6

Context-Tiered Identity Access Control

Shareable / Challenge / Never Data Tiers

Project: personal-agents

Overview

When a personal AI agent has access to everything about you, you need a principled system for deciding what information it can share with the outside world and under what conditions. The personal-agents project implements a three-tier identity access control system that classifies every piece of personal data into one of three tiers: Shareable (safe to surface publicly), Challenge (requires identity verification), and Never (never leaves the vault under any circumstances).

This architecture mirrors real-world identity verification: your name and city appear on business cards (Shareable), your date of birth verifies you with a bank (Challenge), and your SSN is never transmitted or spoken aloud (Never). The AI agent applies these rules automatically when composing responses, redacting Never-tier fields and requiring verification before surfacing Challenge-tier data.

How It Works

- Every personal data field is classified into one of three tiers at schema definition time
- Shareable tier includes name, job title, city - information that appears on public profiles
- Challenge tier includes date of birth, phone, email - requires identity verification
- Never tier includes SSN, passwords, API keys - never returned regardless of context
- Context strings (public, verified, admin) determine which tiers are accessible at runtime
- get_allowed_fields() filters any data dict through the tier rules before returning
- The AI agent passes all outgoing data through this filter before including in responses

identity/tiers.py - Three-tier field classification and access control

```python
class IdentityTier:
    SHAREABLE = "shareable"  # name, job, city
    CHALLENGE = "challenge"  # DOB, phone, email
    NEVER     = "never"      # SSN, passwords, keys

FIELD_TIERS = {
    "full_name": IdentityTier.SHAREABLE,
    "city":      IdentityTier.SHAREABLE,
    "email":     IdentityTier.CHALLENGE,
    "phone":     IdentityTier.CHALLENGE,
    "dob":       IdentityTier.CHALLENGE,
    "ssn":       IdentityTier.NEVER,
    "api_keys":  IdentityTier.NEVER,
}

def get_allowed_fields(context: str, fields: dict) -> dict:
    if context == "public":
        allowed = {IdentityTier.SHAREABLE}
    elif context == "verified":
        allowed = {IdentityTier.SHAREABLE, IdentityTier.CHALLENGE}
    else:
        return {}  # never release NEVER tier
    return {k: v for k, v in fields.items()
            if FIELD_TIERS.get(k) in allowed}
```

■■ Never Tier Is Absolute

The Never tier admits no exceptions - not for verified users, not for the owner asking through the AI interface, not for admin contexts. Never-tier fields are accessed only through direct non-AI code paths with explicit authentication. The AI agent never sees them.

■ Design Tiers Before You Store Data

Retrofitting a tier system onto an existing data model is painful. Every field you store has implicit access assumptions - make them explicit at schema design time. Future you will thank present you when someone tries to jailbreak your agent into revealing sensitive data.

■■ Common Pitfall: Hardcoding Tier Assignments in Application Logic

If tier assignments are scattered across application code rather than defined in a central FIELD_TIERS dictionary, adding a new sensitive field means hunting through the codebase to find every access control check. Define all tier assignments in one authoritative location at schema definition time. Every access control decision should consult that dictionary — never hard-code tier assumptions inline. A code review can then catch any new field missing a tier assignment.

Why It Matters

Personal AI agents that handle sensitive data are a double-edged sword: vastly more useful when they know everything about you, but that knowledge becomes a liability if the agent can be manipulated into revealing it. The tiered access model draws a clear line between what the agent knows internally and what it is allowed to surface externally, providing principled defense against both accidental disclosure and intentional prompt injection attacks.

Chapter 7

Call Memory Learning System

Per-Contact Intelligence Across Sessions

Project: personal-agents

Overview

A personal AI agent that forgets every interaction is like an assistant with amnesia. The Call Memory Learning System persists structured intelligence about every contact the agent interacts with: their preferences, history of past calls, recurring topics, and sentiment trends. This enables the agent to provide personalized, context-aware responses that improve with every interaction rather than treating every call as the first.

The system stores two types of data: contact profiles (persistent, cumulative information about each person) and call logs (timestamped records of each interaction with extracted summaries and sentiment). Before processing any call, the agent retrieves the contact context and injects it into the conversation prompt. After each call, it updates the profile with new learnings. Over time, the agent develops a rich model of each contact.

How It Works

- Contact profiles store cumulative data: name, preferences, total call count, last interaction summary
- Call logs record each interaction with timestamp, AI-generated summary, and sentiment
- Before each call, get_contact_context() retrieves the last 10 call logs and current profile
- The context is injected into the agent system prompt as structured JSON
- After each call, update_after_call() atomically updates the profile and inserts a new log
- Sentiment tracking enables the agent to adapt its communication style per contact
- SQLite with aiosqlite ensures non-blocking database operations in the async event loop

memory/call_memory.py - Contact context retrieval and post-call update

```python
class CallMemorySystem:
    def __init__(self, db):
        self.db = db

    async def get_contact_context(self, phone: str):
        contact = await self.db.get(Contact, phone)
        if not contact:
            return ContactContext(is_new=True)
        recent = await self.db.execute(
            "SELECT summary, sentiment FROM call_logs"
            " WHERE phone=? ORDER BY created_at DESC LIMIT 10",
            (phone,)
        )
        return ContactContext(
            is_new=False,
            name=contact.name,
            call_count=contact.total_calls,
            preferences=contact.preferences,
            recent_summaries=[r[0] for r in recent.fetchall()],
        )

    async def update_after_call(self, phone: str, summary: str, sentiment: str):
        await self.db.execute(
            "INSERT INTO call_logs (phone, summary, sentiment) VALUES (?,?,?)",
            (phone, summary, sentiment)
        )
        await self.db.execute(
            "UPDATE contacts SET last_summary=?, total_calls=total_calls+1"
            " WHERE phone=?", (summary, phone)
        )
```

■ Inject Memory as Structured JSON

Do not paste raw conversation logs into the prompt. Extract and structure key learnings: name, preferences, last call topic, sentiment trend. Structured context is more token-efficient and easier for the model to use than raw transcripts. Target under 200 tokens of context per contact.

■ Learning Compounds Over Time

A contact who always asks about flights has that preference stored after the first call. By the fifth call, the agent proactively checks flight prices before they even ask. This is the compound interest of persistent memory - small learnings accumulate into genuinely useful personalization.

Common Pitfall: Injecting Too Much Memory Context

Injecting the last 10 call summaries works fine for contacts with 10 interactions. After 100 interactions, injecting 10 verbose summaries consumes 2000+ tokens of context window that could be used for the model's response. Implement a two-level memory system: detailed recent context for the last 3 interactions, and a compressed cumulative summary for older history. Regenerate the cumulative summary every 5 new interactions via a dedicated summarization LLM call.

Why It Matters

Memory is the foundation of genuine personal AI assistance. Without it, every interaction is a cold start with no context. With it, the agent learns your contacts, preferences, and patterns, becoming more useful with each interaction. The Call Memory System is the difference between a chatbot that answers questions and an AI assistant that actually knows you and your world.

Chapter 8

Fraud Detection in AI Voice Calls

Real-Time Scam Detection and Risk Scoring

Project: personal-agents

Overview

AI-generated voice calls have become a primary vector for sophisticated fraud. Criminals use voice cloning to impersonate bank representatives, government officials, and family members. The personal-agents project implements a two-layer fraud detection system that analyzes call transcripts in real-time: a fast pattern-matching layer catches known scam phrases instantly, while a deep LLM analysis layer identifies novel fraud patterns that pattern matching would miss.

The system processes live call transcripts through a fraud analysis pipeline running asynchronously alongside the conversation. When risk thresholds are exceeded, the agent can interrupt the call, alert the user, or automatically end the connection. The LLM layer is particularly powerful because it understands context: a legitimate bank calling about suspicious activity uses fundamentally different language patterns than a scammer pretending to be the same bank.

How It Works

- A fast pattern-matching layer checks transcripts against a curated database of known scam phrases
- The LLM analysis layer processes the full transcript context to identify novel fraud patterns
- Risk scoring combines pattern hits, LLM risk level, and caller ID reputation into a composite score
- Threshold-based triggers activate different responses: alert, interrupt, or end call automatically
- Real-time processing enables mid-call interventions before the user is defrauded
- Caller ID reputation lookups cross-reference known scam number databases
- All detected fraud attempts are logged and fed back to improve pattern databases over time

fraud/detector.py - Two-layer fraud detection with LLM analysis

```python
SCAM_PATTERNS = [
    "your account will be suspended",
    "verify your social security",
    "immediate action required",
    "wire transfer today",
    "gift card payment",
]

class FraudDetector:
    async def analyze(self, transcript: str, caller_id: str):
        # Layer 1: Fast pattern matching
        hits = [p for p in SCAM_PATTERNS if p in transcript.lower()]

        # Layer 2: LLM deep analysis
        prompt = (
            "Analyze for fraud indicators. Return JSON:\n"
            "{risk_level, indicators, recommendation}\n"
            "Transcript: " + transcript
        )
        analysis = json.loads(await self.llm.generate(prompt))
        return FraudReport(
            risk_level=analysis["risk_level"],
            pattern_hits=hits,
            indicators=analysis["indicators"],
            recommendation=analysis["recommendation"],
        )
```

■

Two-Layer Defense Is Essential

Pattern matching alone misses novel scams - fraudsters constantly adapt their scripts. LLM analysis alone is too slow for real-time detection. The combination provides both speed (pattern matching in milliseconds) and depth (LLM catches context-dependent fraud that simple string matching cannot detect).

■■

Calibrate False Positive Rates Carefully

A fraud detector that flags too many legitimate calls trains users to ignore warnings. Calibrate thresholds with real call data. A 1% false positive rate is acceptable. A 10% rate destroys trust in the entire system.

■ Common Pitfall: Static Pattern Databases Go Stale Within Weeks

Scammers actively monitor fraud detection systems and adapt their scripts within days of a pattern becoming widely known. A static SCAM_PATTERNS list that is never updated becomes ineffective quickly. Implement a feedback loop: every confirmed fraud report (user-verified) should automatically extract new trigger phrases via LLM and add them to the pattern database. The system must improve with every detected fraud case, or it degrades relative to evolving scam techniques.

Why It Matters

Phone fraud causes billions in losses annually and disproportionately affects vulnerable populations. An AI agent with real-time fraud detection becomes a trusted guardian recognizing scam patterns across thousands of variations. As AI-generated voice calls become indistinguishable from human callers, the only scalable defense is AI-powered detection that analyzes context and intent rather than just surface-level patterns.

Chapter 9

CommandCore - 20-Service Telegram Hub

Orchestrating a Full Personal OS via Telegram

Project: commandcore

Overview

CommandCore is a personal operating system delivered through Telegram. Rather than using 20 separate apps for weather, news, price alerts, system monitoring, backup, and calendar sync, CommandCore unifies all of them behind a single Telegram bot interface. The orchestration engine launches 12 concurrent background services on startup and exposes each as a Telegram command, creating a unified command-and-control interface for daily life.

The architecture follows a startup orchestration pattern: connectDb() initializes the database connection, startWorker() boots the BullMQ job queue, then Promise.all() launches all 12 background services concurrently. Each service runs its own internal scheduler independently. The Telegram bot and HTTP server start last, becoming the user-facing interfaces once all services are initialized and ready.

How It Works

- connectDb() establishes the database connection before any service that needs persistence starts
- startWorker() boots the BullMQ background job processor for async task handling
- Promise.all() starts all 12 background services concurrently, reducing total startup time
- Each service has its own internal scheduler, error handling, and retry logic
- createBot() registers each Telegram command with its corresponding handler function
- The HTTP server runs in parallel with the Telegram bot for webhook and API endpoints
- Unhandled promise rejections in any service are caught and logged without crashing the process

src/index.ts - CommandCore startup orchestration with 12 concurrent services

```
async function main() {
  await connectDb()
  const worker = startWorker()
  const app = createServer(config)
  const bot = createBot(config)
  await Promise.all([
    startMorningBriefing(), startCronRunner(), startDailyDigest(),
    startSysMonitor(),      startPingRunner(), startReminderRunner(),
    startRssRunner(),       startPriceWatcher(), startProcessWatcher(),
    startAppointmentNagger(), startBackupRunner(), startCalendarSync(),
  ])
  await Promise.all([startServer(app), startBot(bot)])
}
main().catch(console.error)
```

src/bot/index.ts - Telegram command registration for all services

```
export function createBot(config: AppConfig) {
  const bot = new Bot(config.TELEGRAM_TOKEN)
  bot.command("brief",   briefingHandler)
  bot.command("price",   priceHandler)
  bot.command("news",    newsHandler)
  bot.command("remind",  reminderHandler)
  bot.command("weather", weatherHandler)
  bot.command("rss",     rssHandler)
  bot.command("ping",    pingHandler)
  bot.command("backup",  backupHandler)
  bot.command("cal",     calendarHandler)
  bot.command("monitor", monitorHandler)
  return bot
}
```

■ Promise.all() for Parallel Service Startup

Starting 12 services sequentially with await would take the sum of each initialization time. Promise.all() starts them concurrently, cutting startup time to the slowest single service. This matters for cold starts and process restarts, especially when services have network-dependent initialization.

■■ Each Service Owns Its Failure Mode

Background services that crash should restart themselves, not crash the entire process. Wrap each service in a try/catch with exponential backoff retry logic. CommandCore has been running continuously for months because individual service failures never propagate to the host process.

■■ Common Pitfall: One Crashing Service Killing the Entire Hub

If any Promise in your startup Promise.all() throws and is not caught, the entire process crashes — including the 11 services that started successfully. Wrap each service startup in its own try/catch and return a resolved promise on failure, marking that service as degraded. Log the failure with full context. A partially-functional hub that serves 11 of 12 services is infinitely more useful than a crashed hub serving zero.

Why It Matters

Telegram as an interface for a personal operating system offers push notifications, multi-device sync, persistent history, and authentication for free. CommandCore demonstrates that a well-orchestrated system of small services, each doing one thing well, beats monolithic applications for personal tooling. The startup orchestration pattern shown here scales from 12 services to 50 without architectural changes.

Chapter 10

BullMQ Job Queue System

Background Task Processing with Redis

Project: commandcore

Overview

Long-running tasks in a web server or Telegram bot block the event loop and cause timeouts, dropped messages, and degraded user experience. BullMQ solves this by moving heavy work into a separate worker process that processes jobs from a Redis-backed queue. CommandCore uses BullMQ to handle RSS scraping, morning briefing generation, database backups, and price fetching without blocking the bot from responding to user commands.

BullMQ provides enterprise-grade job queue features out of the box: retry logic with exponential backoff for failed jobs, rate limiting to prevent API abuse, cron-based recurring jobs, job concurrency control, and priority queuing. The Redis backend ensures jobs survive process restarts - if the worker crashes mid-backup, Redis remembers and the job is retried when the worker restarts. This durability separates BullMQ from simple setInterval() patterns.

How It Works

- A BullMQ Queue instance backed by Redis stores pending, active, and completed jobs durably
- A Worker pulls jobs from the queue and dispatches them to type-specific handler functions
- concurrency: 3 means the worker processes up to 3 jobs simultaneously without blocking
- The rate limiter caps at 10 jobs per second to prevent overwhelming downstream APIs
- Recurring jobs use cron expressions: RSS scraping every 6 hours, briefing every morning at 7am
- Failed jobs retry 3 times with exponential backoff before entering the failed state
- Job events (completed, failed, stalled) are observable via QueueEvents for monitoring

src/jobs/worker.ts - BullMQ worker with job routing and rate limiting

```
import { Queue, Worker } from "bullmq"
import { Redis } from "ioredis"

const connection = new Redis(process.env.REDIS_URL!)
export const jobQueue = new Queue("jobs", { connection })

export function startWorker() {
  return new Worker("jobs", async (job) => {
    switch (job.name) {
      case "scrape-rss":    return await scrapeRSS(job.data)
      case "send-brief":    return await sendMorningBrief(job.data)
      case "backup-db":     return await backupDatabase(job.data)
      case "fetch-prices":  return await fetchPrices(job.data)
      default: throw new Error("Unknown job: " + job.name)
    }
  }, {
    connection,
    concurrency: 3,
    limiter: { max: 10, duration: 1000 },
  })
}

// Enqueue a recurring job
await jobQueue.add("scrape-rss", { feedUrl }, {
  repeat: { pattern: "0 */6 * * *" },  // every 6 hours
  attempts: 3,
  backoff: { type: "exponential", delay: 5000 },
})
```

■ Redis Persistence Is the Key Advantage

Unlike in-memory queues, BullMQ jobs survive process restarts. If your server crashes during a database backup, the job retries when the process restarts. This durability is non-negotiable for production background tasks that must complete exactly once.

■ Match Concurrency to Your Resources

concurrency: 3 works well for I/O-bound tasks like API calls. CPU-bound tasks (image processing, PDF generation) should use concurrency: 1 to avoid starving the Node.js event loop. Monitor worker CPU and memory to find the right setting for each job type.

■■ Common Pitfall: Not Setting maxStalledCount

If a worker crashes mid-job, BullMQ marks the job as stalled after a timeout and re-enqueues it for retry. Without maxStalledCount configured, a consistently-crashing job re-enqueues indefinitely, consuming all worker capacity in a crash loop. Set maxStalledCount: 2 so that a job that stalls multiple times moves to the failed state where you can inspect its error and data, rather than retrying forever and starving other jobs in the queue.

Why It Matters

Every production application eventually needs background job processing. setInterval() breaks under load, cron jobs lose state on restart, and serverless functions have timeout limits. BullMQ provides the right primitives for durable, scalable background processing: reliable delivery, automatic retry, rate limiting, and observability. CommandCore has been running its background jobs for months with zero lost jobs thanks to this architecture.

PART 3

Playwright & Browser Automation

Chapter 11

AgentBrowser - Async Playwright Wrapper for AI Agents

Headless Browser Automation with AI Control

Project: personal-agents

Overview

AI agents that can only process text are fundamentally limited. An agent that can browse the web, fill forms, click buttons, and extract data from JavaScript-rendered pages is dramatically more capable. The AgentBrowser class in personal-agents is an async Playwright wrapper that gives AI agents a controlled, session-persistent browser interface with structured output extraction, error recovery, and screenshot capture for visual confirmation.

Playwright is the modern standard for browser automation: it handles JavaScript-rendered content, network request interception, multi-tab coordination, and browser context management. The AgentBrowser wraps these capabilities behind a simple async API that AI agents can call without managing the browser lifecycle directly. Sessions persist across calls, enabling multi-step workflows like login followed by data extraction.

How It Works

- A single Playwright browser instance is created at startup and shared across all agent requests
- Browser contexts isolate sessions between different users or workflows with separate cookies and storage
- Each page interaction returns structured data rather than raw HTML for easier AI processing
- Screenshot capture at key moments provides visual confirmation and debugging capability
- Network request interception can block ads and trackers for faster page loads
- Retry logic with exponential backoff handles flaky page loads and intermittent timeouts
- Async context manager pattern ensures browser resources are properly cleaned up on completion

browser/agent_browser.py - Async Playwright wrapper with session management

```python
class AgentBrowser:
    def __init__(self):
        self._browser = None
        self._context = None

    async def __aenter__(self):
        pw = await async_playwright().start()
        self._browser = await pw.chromium.launch(headless=True)
        self._context = await self._browser.new_context(
            user_agent="Mozilla/5.0 (compatible; AgentBot/1.0)",
            viewport={"width": 1280, "height": 720},
        )
        return self

    async def __aexit__(self, *args):
        await self._context.close()
        await self._browser.close()

    async def navigate_and_extract(self, url: str, selector: str) -> list[str]:
        page = await self._context.new_page()
        await page.goto(url, wait_until="networkidle")
        elements = await page.query_selector_all(selector)
        return [await el.text_content() for el in elements]
```

■ networkidle vs domcontentloaded

Use wait_until="networkidle" for JavaScript-heavy SPAs that load data asynchronously. Use "domcontentloaded" for static pages where DOM is sufficient. Incorrect wait conditions are the primary cause of Playwright timeouts in production automation.

■ Browser Contexts Are Lightweight

Creating a new browser context costs much less than spawning a new browser instance. Use separate contexts for different users or sessions that need isolated cookies and local storage, and reuse the same browser instance across all of them.

■■ Common Pitfall: Browser Context Leaks on Exception

If an exception occurs inside your Playwright automation before the browser.close() call, the browser process keeps running in the background, consuming memory and file descriptors. Always use try/finally or the async context manager pattern (async with async_playwright() as pw) to guarantee cleanup on both success and failure paths. A browser process leak of even 50MB per context will exhaust server memory within hours on a high-traffic automation system.

Why It Matters

Browser automation is what transforms an AI agent from a sophisticated chatbot into a true autonomous agent. With a browser, the agent can book flights, fill government forms, extract prices from competitor sites, monitor inventory levels, and interact with any web interface that a human can. The AgentBrowser abstraction makes these capabilities accessible to any agent with a simple async API while handling all the Playwright lifecycle complexity internally.

Chapter 12

Playwright Autotrader Scraper

JS-Rendered Automotive Listings Extraction

Project: car-deal-researcher

Overview

Autotrader and similar automotive listing sites load their inventory dynamically using JavaScript frameworks that render content after the initial page load. Traditional HTTP requests using requests or httpx only retrieve the HTML skeleton without any listing data. The car-deal-researcher project uses Playwright to fully render these JavaScript-heavy pages before extracting structured listing data, enabling comprehensive price research and deal identification across thousands of listings.

The scraper handles the complete extraction pipeline: navigating to search results with configurable filters, waiting for JavaScript rendering to complete, handling pagination across multiple result pages, extracting structured data from each listing card, and normalizing the data into a consistent schema. Anti-bot detection is addressed with realistic user agent strings, randomized delays, and browser fingerprint settings.

How It Works

- Playwright launches a full Chromium browser with JavaScript enabled to render dynamic content
- Search filters (make, model, year range, price, mileage) are applied via URL parameters and form inputs
- wait_for_selector() ensures listing cards are fully rendered before extraction begins
- A pagination loop follows "next page" links and accumulates listings across all result pages
- Structured extraction maps DOM selectors to a standardized ListingSchema with Pydantic validation
- Randomized delays between page navigations (1-3 seconds) mimic human browsing patterns
- Extracted listings are stored in SQLite with deduplication by VIN to prevent double-counting

scrapers/autotrader.py - Playwright-based JS-rendered listing extraction

```python
async def scrape_autotrader(
    make: str, model: str, max_price: int, zip_code: str
) -> list[Listing]:
    async with async_playwright() as pw:
        browser = await pw.chromium.launch(headless=True)
        page = await browser.new_page()
        url = (
            f"https://www.autotrader.com/cars-for-sale"
            f"/{make}/{model}?zip={zip_code}"
            f"&maxPrice={max_price}&searchRadius=100"
        )
        await page.goto(url, wait_until="networkidle")
        await page.wait_for_selector("[data-cmp=listingContainer]")
        listings = []
        while True:
            cards = await page.query_selector_all("[data-cmp=listingCard]")
            for card in cards:
                price = await card.query_selector("[data-cmp=price]")
                title = await card.query_selector("[data-cmp=cardTitle]")
                listings.append(Listing(
                    price=int((await price.text_content()).replace("$","").replace(",","")),
                    title=await title.text_content(),
                ))
            next_btn = await page.query_selector("[aria-label=Next]")
            if not next_btn: break
            await next_btn.click()
            await asyncio.sleep(random.uniform(1.5, 3.0))
        await browser.close()
        return listings
```

■ JavaScript Rendering Is Non-Negotiable for SPAs

Autotrader, Zillow, LinkedIn, and most modern platforms render their content via React or Vue after page load. A requests.get() on these URLs returns an empty shell. Playwright is the only way to get the actual content without reverse-engineering their APIs.

■■ Respect Rate Limits and robots.txt

Web scraping exists in a legal and ethical gray area. Always check robots.txt, add realistic delays between requests, and avoid scraping personal data. Commercial-scale scraping may violate terms of service. Use scraped data for personal research only.

Why It Matters

Car shopping without comprehensive price research means paying thousands more than necessary. The Autotrader scraper enables programmatic research across entire regional markets - something impossible to

do manually across hundreds of listings. The same pattern applies to real estate (Zillow), job listings (LinkedIn, Indeed), and any domain where JavaScript-rendered data needs systematic extraction for analysis.

Chapter 13

Bank Portal Automation with Session Persistence

Secure Login and Data Extraction from Financial Portals

Project: rent-tracker

Overview

Bank and financial institution portals are among the most challenging automation targets: they use sophisticated bot detection, multi-factor authentication, session timeouts, and dynamically generated form tokens. The rent-tracker project implements a bank portal automation system that handles the complete login sequence, persists sessions across multiple data extraction cycles using Playwright storage state, and gracefully handles session expiration with automatic re-authentication.

The automation extracts rent payment history, balance information, and transaction records from the bank portal and reconciles them against expected rent payments tracked in the local database. This eliminates the need to manually log into the bank portal to verify payments, enabling automated reconciliation and alerting when expected payments are not received within the expected window.

How It Works

- Playwright storage state serializes cookies and localStorage to disk after successful login
- Subsequent sessions restore from the saved state, bypassing the login sequence if still valid
- Session validity is checked by attempting to access an authenticated page and checking the response
- MFA codes from authenticator apps are entered via secure prompts integrated with the workflow
- Dynamic CSRF tokens and form challenges are handled by extracting them from the page before submission
- Transaction data is extracted from HTML tables or JavaScript state using multiple selector strategies
- All credentials are stored in encrypted vault files, never in plain text or environment variables

bank/session.py - Bank portal with session persistence and automatic re-auth

```python
class BankPortalSession:
    STATE_FILE = "bank_session.json"

    async def get_authenticated_page(self):
        pw = await async_playwright().start()
        browser = await pw.chromium.launch(headless=True)
        # Try to restore saved session
        if Path(self.STATE_FILE).exists():
            ctx = await browser.new_context(
                storage_state=self.STATE_FILE
            )
            page = await ctx.new_page()
            await page.goto(BANK_URL + "/dashboard")
            if "dashboard" in page.url:  # session still valid
                return page
            await ctx.close()
        # Fresh login
        ctx = await browser.new_context()
        page = await ctx.new_page()
        await page.goto(BANK_URL + "/login")
        await page.fill("#username", self.username)
        await page.fill("#password", self.password)
        await page.click("#login-btn")
        await page.wait_for_url("**/dashboard")
        # Persist session for future use
        await ctx.storage_state(path=self.STATE_FILE)
        return page
```

■ Storage State Is the Session Persistence Key

Playwright storage state serializes all cookies, localStorage, and sessionStorage to a JSON file. Restoring from this file is equivalent to reopening the browser with an active session - no login sequence needed. This cuts automation time from 15 seconds (login) to 2 seconds (restore) for every run.

■■ Never Store Credentials in Plain Text

Bank automation handles highly sensitive credentials. Always encrypt credentials at rest using Fernet or similar symmetric encryption (see Chapter 36). Never store credentials in environment variables for production systems - they appear in process listings, log files, and crash dumps.

Why It Matters

Rent-tracker transforms a monthly manual chore - logging into the bank portal to verify rent payments arrived - into an automated reconciliation system that runs daily. When a payment is late or missing, the system sends an immediate alert rather than waiting for the landlord to discover the issue during the next login. The same session persistence pattern applies to any automation that needs to maintain authenticated state across multiple extraction cycles.

Chapter 14

Playwright Scheduling Agent

Automated Appointment Booking on Zocdoc and Clinics

Project: personal-agents

Overview

Healthcare appointment booking is one of the most tedious scheduling tasks: finding available slots across multiple providers, comparing availability with your calendar, and completing the booking form with insurance and personal information. The Playwright Scheduling Agent automates this entire workflow - searching for available appointments across multiple booking platforms, cross-referencing with Google Calendar for conflicts, and completing the booking automatically when a suitable slot is found.

The agent handles the specific challenges of medical booking platforms: dynamic calendar rendering with available slot indicators, insurance verification forms, provider selection, and confirmation code capture. For platforms without an API, Playwright handles form interaction directly. The agent also sends confirmation details to the user via Telegram and adds the appointment to Google Calendar automatically.

How It Works

- The agent searches multiple booking platforms (Zocdoc, clinic portals) with configurable search criteria
- Available slots are extracted and cross-referenced against the user Google Calendar via API
- Preferred slot criteria (morning vs afternoon, day of week, provider) are applied as filters
- Playwright fills the booking form with pre-stored personal and insurance information
- Confirmation codes and appointment details are captured from the confirmation page
- A Telegram notification is sent with full appointment details and cancellation links
- The appointment is added to Google Calendar with provider address and contact info

agents/scheduling.py - Zocdoc appointment booking with slot selection

```python
async def book_zocdoc_appointment(
    specialty: str, date_range: tuple, insurance: str
) -> AppointmentResult:
    async with AgentBrowser() as browser:
        page = await browser.navigate(
            f"https://www.zocdoc.com/search?specialty={specialty}"
            f"&insurance={insurance}"
        )
        await page.wait_for_selector(".provider-card")
        providers = await page.query_selector_all(".provider-card")
        for provider in providers[:5]:
            slots = await provider.query_selector_all(".slot-btn:not(.disabled)")
            for slot in slots:
                time_text = await slot.text_content()
                if is_preferred_time(time_text, date_range):
                    await slot.click()
                    await fill_patient_form(page, PATIENT_INFO)
                    await page.click("#confirm-booking")
                    confirm = await page.wait_for_selector(".confirmation-code")
                    code = await confirm.text_content()
                    return AppointmentResult(code=code, time=time_text)
```

■ Handle Dynamic Calendar Rendering

Medical booking calendars often render available slots via AJAX after initial page load. Use wait_for_selector() with the specific slot element class rather than wait_for_load_state() to ensure the calendar is fully populated before extraction begins.

■■ Store Patient Information Encrypted

The scheduling agent stores insurance information, date of birth, and contact details. All of this is sensitive health-adjacent data. Store it in the Fernet-encrypted vault described in Chapter 36, not in plain text files or environment variables.

Why It Matters

Healthcare scheduling is a multi-step, multi-platform task that consumes significant time and cognitive load. The scheduling agent handles the entire workflow from search to confirmation, taking 30 seconds to accomplish what takes humans 15-20 minutes of clicking through calendar interfaces and form fields. As AI agents become more capable at handling complex multi-step browser workflows, this pattern becomes the foundation for automating any high-friction scheduling process.

PART 4

Real-Time Streaming & WebSockets

Chapter 15

Binance WebSocket Price Feed with Reconnection

Real-Time Crypto Price Streaming and Backoff

Project: btc-sniper

Overview

Financial trading systems require sub-second price data that REST API polling fundamentally cannot provide — and the gap is not marginal. Polling Binance REST for BTC price data every second generates 86,400 HTTP requests per day per symbol, most returning no new data since the last poll. Each request takes 50-150ms of round-trip time, so your prices are already stale before you even process them. WebSockets invert this model entirely: you open a single persistent TCP connection, and the exchange pushes every executed trade to your process within 50ms of occurrence. One Binance WebSocket connection delivers continuous, push-based tick data that replaces thousands of REST calls per hour. The performance difference is so large that any latency-sensitive trading logic should never use REST polling for price data.

The BinanceFeed class maintains a rolling deque of the most recent 1000 price points. The critical design question is: what happens during reconnection? The deque preserves the last prices received before the disconnect, but trades continue to occur on the exchange during the reconnection window. This is unavoidable data loss — the strategy engine must treat a gap in timestamps as a signal to reduce confidence in its signals or halt new entries until the feed is confirmed stable. The exponential backoff (1s, 2s, 4s, up to 60s) serves a second purpose beyond being polite: Binance imposes IP bans for aggressive connection attempts during maintenance windows, which are precisely the moments when data reliability matters most.

How It Works

- A websockets client connects to the Binance trade stream endpoint — a raw TCP connection that stays open for hours without re-authentication
- Each incoming message is parsed from JSON and stored in a deque with the current UTC timestamp appended
- The exponential backoff resets to 1 second on successful connection — confirming the exchange is reachable resets all penalty state
- Maximum backoff is capped at 60 seconds to ensure eventual recovery: never wait longer than one minute before attempting reconnect

- The running flag enables clean shutdown by exiting the outer while loop — no orphaned connections or background threads
- A deque with maxlen=1000 acts as a fixed-size circular buffer — oldest entries are discarded automatically as new ones arrive
- get_latest_prices() returns the last N entries synchronously — safe to call from the strategy engine without async overhead

feeds/binance_ws.py - WebSocket feed with exponential backoff reconnection

```
class BinanceFeed:
    TRADE_STREAM = "wss://stream.binance.com:9443/ws/btcusdt@trade"

    def __init__(self):
        self._prices = deque(maxlen=1000)
        self._running = True

    async def start(self):
        backoff = 1
        while self._running:
            try:
                async with websockets.connect(self.TRADE_STREAM) as ws:
                    backoff = 1  # reset on successful connection
                    async for msg in ws:
                        data = json.loads(msg)
                        self._prices.append({
                            "price": float(data["p"]),
                            "qty":   float(data["q"]),
                            "time":  datetime.utcnow(),
                        })
            except Exception as e:
                print(f"WS error: {e}, retrying in {backoff}s")
                await asyncio.sleep(min(backoff, 60))
                backoff *= 2

    def get_latest_prices(self, n: int = 100) -> list:
        return list(self._prices)[-n:]
```

■■ Exponential Backoff Is Non-Negotiable

A feed that reconnects immediately on failure can send thousands of connection attempts per minute during an exchange outage. This triggers IP bans and makes the outage worse. Exponential backoff protects both the exchange and your IP reputation while ensuring eventual recovery.

■ Use deque with maxlen for Memory Safety

A simple list accumulating ticks runs indefinitely and eventually exhausts memory. deque(maxlen=1000) automatically discards the oldest elements as new ones arrive, providing a fixed-size circular buffer at negligible cost.

■■ Common Pitfall: Silent Data Gaps After Reconnection

After a WebSocket reconnects, your code receives the next incoming trade — but there may be hundreds of trades that occurred during the disconnection window that you never saw. The deque appears to have continuous data, but there is a silent time gap between the last pre-disconnect timestamp and the first post-reconnect timestamp. Always log disconnect and reconnect timestamps alongside the price buffer, and expose a is_data_fresh(max_gap_seconds) method so your strategy engine can detect gaps and reduce signal confidence accordingly.

Why It Matters

Reliable real-time market data is the foundation of any trading system — a strategy running on stale or gapped price data makes wrong decisions without knowing it. The reconnection logic in BinanceFeed ensures that brief network interruptions (common in both home and cloud environments) do not translate into strategy blindness. The same WebSocket-with-backoff pattern applies to any domain requiring real-time push data: stock tickers, sports scores, IoT sensor feeds, and social media streams. Mastering this pattern means you can build reliable real-time data pipelines for any source that supports WebSocket connections.

Chapter 16

Multi-Signal Ensemble Trading System

Order Book, Volume, and Funding Rate Signals

Project: btc-sniper

Overview

No single market indicator is reliably predictive on its own. Order book imbalance can signal short-term price direction but is easily manipulated. Volume spikes can indicate genuine momentum or wash trading. Funding rates reveal leveraged positioning but lag actual price moves. The btc-sniper Multi-Signal Ensemble Trading System combines these three signal types into a weighted scoring system that generates higher-confidence trade signals than any single indicator could produce.

The ensemble approach works by computing a normalized score (-1 to +1) for each signal independently, then computing a weighted average. Weights are calibrated based on historical signal performance during different market regimes. When the ensemble score exceeds a configurable threshold, a trade signal is generated. The system also implements a signal stability filter that requires the score to remain above threshold for multiple consecutive samples before triggering, reducing whipsaw signals.

How It Works

- Order book imbalance computes the ratio of bid volume to ask volume at the top 5 price levels
- Volume signal compares current 5-minute volume against a 20-period rolling baseline
- Funding rate signal classifies the current perpetual swap funding rate as bullish/neutral/bearish
- Each signal is normalized to a -1 (bearish) to +1 (bullish) scale before combination
- Weighted average combines signals: order_book (0.4), volume (0.3), funding (0.3) by default
- A stability filter requires score above threshold for 3 consecutive samples before triggering
- Position sizing uses the Kelly criterion (Chapter 21) applied to the ensemble confidence score

signals/ensemble.py - Multi-signal weighted scoring system

```python
class EnsembleSignal:
    def __init__(self, weights=(0.4, 0.3, 0.3)):
        self.w_book, self.w_vol, self.w_fund = weights

    def compute(self, book: OrderBook, vol_data: VolumeData,
                funding: float) -> float:
        # Order book imbalance (-1 to +1)
        bid_vol = sum(b.qty for b in book.bids[:5])
        ask_vol = sum(a.qty for a in book.asks[:5])
        book_signal = (bid_vol - ask_vol) / (bid_vol + ask_vol)

        # Volume signal (-1 to +1)
        vol_ratio = vol_data.current / (vol_data.baseline + 1e-9)
        vol_signal = min(1.0, (vol_ratio - 1.0)) * (1 if vol_data.bullish else -1)

        # Funding signal (-1 to +1)
        fund_signal = max(-1.0, min(1.0, -funding / 0.001))

        # Weighted ensemble
        return (
            self.w_book * book_signal +
            self.w_vol  * vol_signal  +
            self.w_fund * fund_signal
        )
```

■

Ensemble Beats Single Indicators

Academic research consistently shows that combining uncorrelated signals outperforms any individual signal in terms of Sharpe ratio and maximum drawdown. The key is combining signals that have low correlation to each other - order book imbalance and funding rates are largely independent, making them effective ensemble components.

■■

Calibrate Weights with Historical Data

Default weights of 0.4/0.3/0.3 are starting points, not optimal values. Backtest your signal combination on at least 6 months of historical data and tune weights to maximize Sharpe ratio while controlling maximum drawdown. Weights that work in trending markets often fail in ranging markets.

■■ Common Pitfall: Correlated Signals Defeat the Ensemble Purpose

If order book imbalance and volume signal both spike during the same market events for the same structural reasons, they are correlated — and combining correlated signals provides less diversification benefit than their weights suggest. Before finalizing signal weights, compute the correlation matrix between your signals over 90 days of historical data. Only combine signals with pairwise correlation below 0.3. Higher correlation means you are effectively doubling down on a single market view while believing you are diversifying.

Why It Matters

Ensemble methods are the foundation of most successful quantitative trading strategies. The principle - combining multiple imperfect signals into a more reliable composite - is the same technique used by weather forecasters, medical diagnosticians, and election prediction models. In trading, ensemble signals reduce the false positive rate (whipsaw trades) while maintaining sensitivity to genuine market moves, leading to better risk-adjusted returns.

Chapter 17

FastAPI WebSocket Conversation Server

Real-Time Bidirectional AI Conversation

Project: personal-agents

Overview

HTTP request-response is a poor fit for conversational AI interfaces where multiple turns of dialogue need to flow between client and server without repeated connection setup overhead. FastAPI WebSocket endpoints provide persistent bidirectional connections where the server can push streaming AI responses to the client while the client sends new messages at any time. The personal-agents conversation server uses this pattern to deliver a fluid, real-time chat experience.

The WebSocket server maintains conversation state per connection, injecting the appropriate system prompt and memory context for the connected user. When a message arrives, the server routes it through the multi-agent system (Chapter 5), streams the response back token by token, and updates the conversation memory asynchronously. The clean architecture separates WebSocket lifecycle management, conversation state, and agent execution into distinct concerns.

How It Works

- FastAPI WebSocket endpoint accepts connections at /ws/{user_id} with JWT authentication
- A ConnectionManager class tracks active WebSocket connections and enables targeted broadcasts
- Each connected client maintains its own conversation history and agent context
- Incoming messages are deserialized from JSON and routed to the appropriate agent
- Agent responses are streamed back as individual JSON chunks with type: "chunk" or "done"
- Heartbeat pings every 30 seconds detect stale connections and trigger cleanup
- Reconnection is handled client-side with exponential backoff and conversation state restoration

server/ws.py - FastAPI WebSocket server with streaming agent responses

```python
from fastapi import FastAPI, WebSocket, WebSocketDisconnect

app = FastAPI()

class ConnectionManager:
    def __init__(self):
        self.active: dict[str, WebSocket] = {}

    async def connect(self, user_id: str, ws: WebSocket):
        await ws.accept()
        self.active[user_id] = ws

    def disconnect(self, user_id: str):
        self.active.pop(user_id, None)

@app.websocket("/ws/{user_id}")
async def conversation(ws: WebSocket, user_id: str):
    await manager.connect(user_id, ws)
    ctx = await load_agent_context(user_id)
    try:
        while True:
            msg = await ws.receive_json()
            async def stream_cb(chunk: str):
                await ws.send_json({"type": "chunk", "text": chunk})
            await router.route(msg["text"], ctx, on_chunk=stream_cb)
            await ws.send_json({"type": "done"})
    except WebSocketDisconnect:
        manager.disconnect(user_id)
```

■ Separate Connection State from Conversation State

WebSocket connection objects are ephemeral - they die when the client disconnects. Conversation state (history, context, preferences) must be stored separately in a database or cache. When the client reconnects, load the conversation state by user_id and resume seamlessly.

■■ Handle WebSocket Disconnect Gracefully

Clients disconnect constantly - browser tab closes, mobile apps background, network drops. Always wrap the receive loop in try/except WebSocketDisconnect and clean up connection state in the except block. Failing to clean up leads to memory leaks from accumulated stale connection objects.

Why It Matters

WebSocket-based conversation servers enable the fluid, real-time AI chat experience that users now expect. The architecture scales horizontally by storing all conversation state in Redis or a database rather than in process memory, allowing any server instance to handle reconnections from any client. This stateless server, stateful storage pattern is the foundation of production conversational AI systems at any scale.

Chapter 18

Server-Sent Events (SSE) Streaming in Next.js

Push-Based Real-Time Updates Without WebSockets

Project: market-pulse

Overview

Server-Sent Events (SSE) provide a simpler alternative to WebSockets for scenarios where the server needs to push data to the client unidirectionally. Unlike WebSockets, SSE uses standard HTTP and works through all proxies, load balancers, and CDNs without special configuration. The market-pulse project uses SSE to stream live market score updates, news alerts, and indicator changes to the dashboard without requiring WebSocket infrastructure.

Next.js App Router route handlers support SSE natively through the Response API with a ReadableStream body and appropriate headers. The server keeps the connection open and writes data to the stream as new information arrives, with each event following the SSE protocol format. Clients use the standard EventSource API to receive these updates and update the UI reactively.

How It Works

- The route handler returns a Response with Content-Type: text/event-stream to initiate SSE
- A ReadableStream controller stays open and writes data as new events arrive
- Each SSE message follows the protocol: data: {json_payload} followed by two newlines
- The EventSource API on the client automatically reconnects if the connection drops
- Heartbeat comment lines (: ping) every 15 seconds keep the connection alive through proxies
- The client EventSource object emits onmessage events for each data line received
- Connection cleanup uses the controller close() method when the client disconnects

app/api/market-stream/route.ts - SSE endpoint with heartbeat and cleanup

```
// app/api/market-stream/route.ts
export async function GET() {
  const encoder = new TextEncoder()
  const stream = new ReadableStream({
    async start(controller) {
      const send = (data: object) => {
        controller.enqueue(
          encoder.encode("data: " + JSON.stringify(data) + "\n\n")
        )
      }
      // Send initial state
      send({ type: "init", score: await getMarketScore() })
      // Stream updates every 30 seconds
      const interval = setInterval(async () => {
        send({ type: "update", score: await getMarketScore() })
      }, 30_000)
      // Heartbeat to keep connection alive
      const heartbeat = setInterval(() => {
        controller.enqueue(encoder.encode(": ping\n\n"))
      }, 15_000)
      // Cleanup on disconnect
      return () => {
        clearInterval(interval)
        clearInterval(heartbeat)
      }
    }
  })
  return new Response(stream, {
    headers: {
      "Content-Type": "text/event-stream",
      "Cache-Control": "no-cache",
      "Connection": "keep-alive",
    }
  })
}
```

■ SSE vs WebSockets - When to Choose Each

SSE is ideal for unidirectional server-to-client streaming: dashboards, notifications, live feeds. WebSockets are needed for bidirectional communication: chat, real-time collaboration, gaming. SSE is simpler to implement, works through all proxies, and is supported natively by all modern browsers without libraries.

■■ Cache-Control: no-cache Is Required

Without Cache-Control: no-cache, CDNs and reverse proxies may buffer the stream or refuse to forward it. This header tells all intermediate infrastructure that the response is a live stream that must not be cached or buffered. Always include it alongside Connection: keep-alive for SSE endpoints.

■■ Common Pitfall: SSE Breaks Behind Nginx Without Explicit Co nfiguration

Nginx buffers HTTP responses by default, which completely breaks SSE — the client receives all events in a single delayed batch instead of a real-time stream. Add 'X-Accel-Buffering: no' to your SSE response headers, or configure 'proxy_buffering off' in your Nginx location block for the SSE endpoint. This is the most common cause of 'it works locally but not in production' bugs with SSE — local development has no reverse proxy, so the buffering issue never manifests until you deploy.

Why It Matters

Live market data dashboards that require users to manually refresh are inferior products. SSE streaming enables dashboards to update automatically as market conditions change, giving users a live view without WebSocket complexity. The simplicity of SSE - standard HTTP, no special infrastructure, automatic reconnection built into EventSource - makes it the right tool for most one-way streaming scenarios in web applications.

PART 5

Financial Data Systems

Chapter 19

FRED API Macro Indicators

Federal Reserve Economic Data Integration

Project: market-pulse

Overview

The Federal Reserve Bank of St. Louis publishes the FRED (Federal Reserve Economic Data) database containing over 800,000 economic time series: GDP growth, inflation (CPI), unemployment rate, Treasury yields, credit spreads, housing starts, consumer confidence, and hundreds of other macro indicators. The market-pulse project integrates with the FRED API to pull these indicators and incorporate them into a comprehensive market health scoring system.

The FRED integration fetches recent observations for each tracked indicator, normalizes them to a common scale, and classifies them as bullish, neutral, or bearish based on configurable thresholds. The resulting classifications feed into the weighted market score engine (Chapter 20), where macro indicators provide the fundamental backdrop against which technical signals are evaluated.

How It Works

- The FRED API is accessed via API key with rate limiting at 120 requests per 60 seconds
- Each indicator is identified by a FRED series ID (e.g., UNRATE for unemployment, DGS10 for 10Y yield)
- Recent observations are fetched with realtime_start and observation_start parameters
- Indicator values are normalized using z-scores relative to their 5-year historical distribution
- Configurable thresholds classify each indicator as bullish (-1 to -0.5), neutral, or bearish (0.5 to 1)
- Indicators are cached locally for 4 hours to respect API rate limits and reduce latency
- Missing or stale data falls back to the last known value rather than breaking the scoring pipeline

data/fred.py - FRED API integration for macro indicators

```python
import httpx
from datetime import datetime, timedelta

FRED_BASE = "https://api.stlouisfed.org/fred/series/observations"
INDICATORS = {
    "UNRATE":   ("unemployment_rate", -1),  # lower is bullish
    "CPIAUCSL": ("cpi_yoy",           -1),  # lower is bullish
    "DGS10":    ("treasury_10y",      -1),  # lower is bullish
    "FEDFUNDS": ("fed_funds_rate",    -1),  # lower is bullish
    "UMCSENT":  ("consumer_sentiment", 1), # higher is bullish
}

async def fetch_fred_indicator(series_id: str) -> float:
    params = {
        "series_id": series_id,
        "api_key": FRED_API_KEY,
        "file_type": "json",
        "sort_order": "desc",
        "limit": 1,
    }
    async with httpx.AsyncClient() as client:
        r = await client.get(FRED_BASE, params=params)
        data = r.json()
        return float(data["observations"][0]["value"])
```

■ FRED Is the Gold Standard for Macro Data

FRED is free, comprehensive, and authoritative - all data comes directly from government statistical agencies. Unlike financial data providers that charge thousands per month, FRED provides institutional-quality economic data at no cost with a free API key and generous rate limits. It is the best macro data source available for individual developers.

■ Cache Macro Data Aggressively

Economic indicators are published monthly or quarterly - there is no need to fetch them more than once per day. Cache all FRED data locally and refresh only when the publication schedule indicates new data is available. This conserves API quota and dramatically reduces dashboard load times.

■ Common Pitfall: FRED Data Has Significant Publication Lag

FRED returns the most recently published value, but economic indicators are published with significant lag. The unemployment rate for May is typically published in early June. If your code fetches the 'latest' unemployment data in late May, it receives April's figure — which may be 6 or more weeks old. Always check the realtime_period_start field in FRED responses to understand when the value was published, not just what observation period it covers. Build publication lag into your macro analysis assumptions.

Why It Matters

Macro economic conditions provide the fundamental context in which all other market signals operate. A bullish technical setup during a recession with rising unemployment and high inflation carries very different implications than the same setup during economic expansion. Integrating FRED data transforms a pure technical trading system into a macro-aware system that adapts its interpretation of signals based on the underlying economic environment.

Chapter 20

Weighted Market Score Engine

0-100 Composite Market Health Score

Project: market-pulse

Overview

Individual market indicators are noisy and can send contradictory signals. A 0-100 composite market health score that aggregates multiple indicators into a single number provides a clearer, more actionable view of overall market conditions. The market-pulse Weighted Market Score Engine combines macro indicators (FRED data), technical indicators (moving averages, RSI, MACD), sentiment indicators (news sentiment, put/call ratio), and market breadth measures into a single composite score updated in real-time.

The engine uses a weighted average of normalized sub-scores, where weights are calibrated to reflect each indicator category's historical predictive power. The score is designed to be interpretable: 0-30 indicates bear market conditions, 31-50 neutral/uncertain, 51-70 bullish conditions, 71-100 strong bull market with potential overextension. Dashboard users see the score along with the contributing factors, enabling transparent signal interpretation.

How It Works

- Each indicator category produces a sub-score normalized to 0-100 using min-max scaling
- Category weights sum to 1.0 and are configurable: macro 0.25, technical 0.35, sentiment 0.20, breadth 0.20
- The composite score is the dot product of category scores and weights
- Score changes exceeding 5 points trigger Telegram alerts with the change details
- Historical scores are stored with timestamps for trend analysis and backtesting
- A regime classifier labels the score range (bear/neutral/bull) based on configurable thresholds
- Confidence interval is computed from indicator agreement: high variance means low confidence

scoring/engine.py - Weighted composite score with confidence and regime

```python
from dataclasses import dataclass

@dataclass
class CategoryScore:
    name: str
    score: float  # 0-100
    weight: float
    indicators: dict

class MarketScoreEngine:
    WEIGHTS = {"macro": 0.25, "technical": 0.35, "sentiment": 0.20, "breadth": 0.20}

    async def compute(self) -> MarketScore:
        macro = await self.score_macro()
        technical = await self.score_technical()
        sentiment = await self.score_sentiment()
        breadth = await self.score_breadth()

        categories = [macro, technical, sentiment, breadth]
        composite = sum(
            c.score * self.WEIGHTS[c.name] for c in categories
        )
        variance = sum(
            (c.score - composite) ** 2 * self.WEIGHTS[c.name]
            for c in categories
        )
        return MarketScore(
            score=round(composite, 1),
            confidence="high" if variance < 200 else "low",
            regime=self.classify_regime(composite),
            categories=categories,
        )
```

■■ Weight Calibration Is Ongoing Work

Default weights based on intuition are a starting point. Backtest your scoring system against historical market returns and adjust weights to maximize the correlation between score and subsequent 3-month returns. Recalibrate weights annually as market structures and indicator effectiveness evolve.

■ Transparency Builds Trust

A score of 67 is meaningless without knowing what drove it. Always expose the contributing factors alongside the composite score. Users who can see that the score dropped from 72 to 64 because inflation surprised to the upside will trust and use the system more than users who see an opaque number.

Common Pitfall: Fixed Weights Fail Across Market Regimes

Weights calibrated during a bull market may perform poorly in a bear market because signal correlations and predictive power shift with market regime. The technical indicator weight that worked in a trending market becomes noise during a crisis. Monitor your composite score's predictive accuracy quarterly and recalibrate weights after major market regime changes — consider training separate weight sets for high-VIX and low-VIX environments as a minimum.

Why It Matters

Composite scoring systems transform complex multi-dimensional market data into actionable, interpretable signals. Rather than juggling a dozen separate indicators that may contradict each other, traders and investors get a single number that summarizes market conditions with full transparency into the contributing factors. The architecture is general enough to apply to any domain that requires multi-indicator aggregation: credit risk scoring, medical diagnosis support, sports analytics, or supply chain health monitoring.

Chapter 21

Kelly Criterion and Expected Value Engine

Superforecaster-Grade Position Sizing

Project: polymarket-bot

Overview

The Kelly Criterion, developed by John Kelly at Bell Labs in 1956, provides the mathematically optimal bet size that maximizes long-run wealth growth given a known edge and win probability. The polymarket-bot implements Kelly sizing alongside expected value calculations as a research project studying algorithmic decision-making in probability-based markets. This chapter documents the implementation as an educational engineering exercise.

The superforecaster engine combines probability estimates from multiple sources (LLM analysis of news, base rate statistics, and market prices) and computes both the Kelly fraction and expected value for research purposes. This code is provided for educational study of probability theory and algorithm design only — it is not financial advice and is not intended for actual trading or investment decisions.

Consider a concrete numerical example to make Kelly intuitive. You believe a political event has a 60% chance of happening. A prediction market offers 2:1 odds — you win $2 for every $1 staked if correct, and lose your $1 stake if wrong. Kelly computes: k = (p * odds - q) / odds = (0.60 * 2 - 0.40) / 2 = (1.20 - 0.40) / 2 = 0.40. Full Kelly says bet 40% of your bankroll. But if your 60% estimate is actually 55% (a plausible calibration error), the true optimal fraction is only (0.55 * 2 - 0.45) / 2 = 0.325 — you would be overbetting by 23%. Half-Kelly says bet 20% regardless of your estimate precision. With a $10,000 bankroll, half-Kelly bets $2,000. If you win (60% of the time), you gain $4,000 net. If you lose (40% of the time), you lose $2,000. Expected value: 0.60 * $4,000 - 0.40 * $2,000 = $1,600 per bet. This is the mathematical case for disciplined, edge-adjusted position sizing.

How It Works

- kelly_fraction() computes the optimal bet size as a fraction of current bankroll
- A negative Kelly fraction indicates no edge exists - position should not be taken
- The half-Kelly heuristic (bet 50% of full Kelly) is used to account for estimation errors
- expected_value() computes average profit per dollar invested given win probability and odds
- Positions require both positive EV and Kelly fraction above 0.02 (2%) to execute
- Probability estimates are averaged across multiple sources and calibrated against historical accuracy
- Maximum position size caps Kelly at 0.10 (10%) to prevent catastrophic ruin from model errors

sizing/kelly.py - Kelly criterion with half-Kelly and EV filtering

```python
def kelly_fraction(p_win: float, odds: float) -> float:
    """Compute Kelly optimal bet fraction.
    odds: decimal odds (2.0 = even money, profit equal to stake)
    """
    q_lose = 1 - p_win
    k = (p_win * odds - q_lose) / odds
    return max(0.0, k)  # never negative

def expected_value(
    p_win: float,
    profit_if_win: float,
    loss_if_lose: float
) -> float:
    """Expected profit per dollar staked."""
    return p_win * profit_if_win - (1 - p_win) * loss_if_lose

def size_position(
    bankroll: float,
    p_win: float,
    decimal_odds: float,
    max_fraction: float = 0.10
) -> float:
    """Compute position size in dollars."""
    k = kelly_fraction(p_win, decimal_odds)
    half_k = k / 2  # half-Kelly for safety
    fraction = min(half_k, max_fraction)
    ev = expected_value(p_win, decimal_odds - 1, 1.0)
    if fraction < 0.02 or ev <= 0:
        return 0.0  # no edge or insufficient Kelly
    return bankroll * fraction
```

■ Always Use Half-Kelly

Full Kelly maximizes long-run growth in theory but requires perfectly calibrated probability estimates - which no one has. Half-Kelly provides 75% of the long-run growth of full Kelly while requiring only half the optimal fraction. The reduction in variance is worth the small growth cost. Many professional traders use quarter-Kelly.

■ Kelly Fraction Below 2% Means No Real Edge

A Kelly fraction of 0.005 (0.5%) means you theoretically have an edge, but it is so small that transaction costs, slippage, and estimation errors will erode it entirely. Set a minimum Kelly threshold (2% is reasonable) below which you do not trade, even if the expected value is technically positive.

■■ Common Pitfall: Overestimating Your Probability Edge

Engineers who build probability estimation models systematically overestimate their out-of-sample accuracy. If your model outputs 65% confidence but is truly only 55% accurate on held-out data, the Kelly fraction computed from 65% will be too aggressive and erode your bankroll even on a positive-EV strategy. Always validate estimates against 100+ tracked predictions with known outcomes. Deflate confidence estimates by 5-10 percentage points as a calibration correction until sufficient evidence accumulates.

Why It Matters

The Kelly Criterion is one of the most important results in applied mathematics connecting probability theory, information theory, and decision-making under uncertainty. Studying its implementation builds intuition for calibration, risk management, and algorithm design. This chapter is purely educational — it is not financial advice. Do not make investment or trading decisions based on this code.

Chapter 22

News Sentiment LLM Classifier

LONG / SHORT / NEUTRAL Signal Generation

Project: news-sniper

Overview

News events move markets. Earnings surprises, Fed announcements, geopolitical events, and executive changes can shift asset prices by 5-20% within minutes of publication. The news-sniper project implements a two-stage news sentiment classification pipeline: first, a fast regex-based filter screens incoming articles for market-relevant keywords; then, a LLM classifier determines whether the news is likely to be LONG, SHORT, or NEUTRAL for the specified asset, along with a confidence score and the key sentences that drove the classification.

The classifier is designed for speed and specificity. The LLM prompt is engineered to produce consistent, structured output that can be parsed reliably, and the system prompt includes current price context to help the model assess relative impact magnitude. Classification latency under 2 seconds enables trading signals that arrive while the news is still fresh and before the market has fully priced in the information.

How It Works

- An RSS feed monitor checks configured financial news sources every 60 seconds for new articles
- A keyword filter screens articles for relevance to the target asset before LLM classification
- The LLM prompt includes the article text, current price, and structured output instructions
- The model returns LONG/SHORT/NEUTRAL, a confidence score, and supporting sentences
- Classification results are stored with full article metadata for backtesting and refinement
- High-confidence signals (above 0.7) trigger immediate Telegram alerts with the article link
- Signal accuracy is tracked against subsequent price movements for continuous model calibration

classifiers/sentiment.py - LLM news sentiment classifier with structured output

```
SENTIMENT_PROMPT = """
Analyze this news article for its likely impact on {ticker} price.
Current price: ${price:.2f}

Article: {article}

Return JSON with keys:
- sentiment: LONG | SHORT | NEUTRAL
- confidence: 0.0 to 1.0
- reasoning: one sentence explaining the classification
- key_quote: most important sentence from the article
"""

async def classify_sentiment(
    article: str, ticker: str, price: float
) -> SentimentResult:
    prompt = SENTIMENT_PROMPT.format(
        ticker=ticker, price=price, article=article[:2000]
    )
    response = await llm.generate(prompt)
    data = json.loads(response)
    return SentimentResult(
        sentiment=data["sentiment"],
        confidence=float(data["confidence"]),
        reasoning=data["reasoning"],
        key_quote=data["key_quote"],
    )
```

■ Truncate Long Articles to 2000 Characters

Most of a news article's market impact is conveyed in the headline and first few paragraphs. Truncating to 2000 characters reduces token cost by 70-80% while losing minimal signal quality. The LLM consistently identifies the key market-moving information from the article lead.

■■ Track Accuracy to Calibrate Confidence Scores

A classifier that says confidence 0.8 for every prediction is not calibrated. Track whether high-confidence predictions actually predict subsequent price movements correctly. If 80% confidence predictions are only right 60% of the time, scale down confidence scores before using them for position sizing.

■ Common Pitfall: Same Article Syndicated Across Multiple RSS Feeds

A major news event generates the same article republished across 10-15 RSS feeds with slightly different headlines. Without deduplication, your sentiment pipeline classifies the same article 15 times and inflates signal strength 15x, making a single story look like an overwhelming market consensus signal. Deduplicate by URL before classification. For near-duplicate articles (same story, different outlets), use cosine similarity on article text to identify the canonical version and classify once.

Why It Matters

News sentiment is one of the few genuine information edges available to systematic traders. By the time a news story is published on major platforms, it is known to millions - but processing its implications quickly and accurately enough to trade on it before the market fully adjusts is where LLM-based classification shines. The key advantage over simpler keyword-based approaches is contextual understanding: the same words can be bullish or bearish depending on context, and LLMs handle this nuance naturally.

Chapter 23

Plaid Banking Integration

Token Exchange and Encrypted Storage

Project: bills-tracker-v2

Overview

Direct bank account integration requires securely handling access tokens that grant read access to sensitive financial data. Plaid provides a standardized OAuth-like flow for bank account linking: the user authenticates with their bank through Plaid's hosted Link interface, and your application receives a one-time public token that you exchange for a permanent access token. The bills-tracker-v2 project implements this complete flow with encrypted token storage and automatic transaction fetching.

The Plaid integration fetches up to 90 days of transaction history on initial connection, then uses incremental updates (new transactions since the last sync cursor) for subsequent refreshes. Transaction data is categorized by Plaid's categorization engine and stored locally with the raw data for custom re-categorization. This enables the affordability assessment engine (Chapter 24) to have complete visibility into the user's financial picture.

How It Works

- Plaid Link iframe handles bank authentication in the browser and returns a one-time public_token
- The server exchanges public_token for access_token using the Plaid /item/public_token/exchange API
- access_token is encrypted with Fernet (Chapter 36) before storage - never stored in plain text
- Transaction sync uses the /transactions/sync API with a cursor for incremental updates
- Item webhook events notify the server when new transactions or connection issues occur
- Re-link flow handles expired or revoked access tokens with user notification
- Institution ID and item ID are stored for display and troubleshooting without exposing the token

integrations/plaid.py - Token exchange and incremental transaction sync

```python
from plaid.api import plaid_api
from plaid.model.item_public_token_exchange_request import (
    ItemPublicTokenExchangeRequest
)

async def exchange_token(public_token: str, user_id: str) -> str:
    request = ItemPublicTokenExchangeRequest(public_token=public_token)
    response = plaid_client.item_public_token_exchange(request)
    access_token = response.access_token
    # Encrypt before storing
    encrypted = cipher.encrypt_text(access_token)
    await db.execute(
        "INSERT INTO plaid_items (user_id, access_token_enc) VALUES (?, ?)",
        (user_id, encrypted)
    )
    return access_token

async def sync_transactions(user_id: str) -> list[Transaction]:
    encrypted = await db.fetchone(
        "SELECT access_token_enc, cursor FROM plaid_items WHERE user_id=?",
        (user_id,)
    )
    access_token = cipher.decrypt_text(encrypted["access_token_enc"])
    cursor = encrypted["cursor"] or ""
    # Fetch new transactions since last sync
    response = plaid_client.transactions_sync(
        TransactionsSyncRequest(access_token=access_token, cursor=cursor)
    )
    return response.added
```

■■ Never Store Plaid Tokens in Plain Text

A Plaid access_token grants read access to a user's complete banking history. Treat it like a password. Encrypt it with field-level encryption before database storage, never log it, and never include it in error messages or stack traces. Use the Fernet vault pattern from Chapter 36.

■ Use Webhooks for Real-Time Transaction Updates

Polling the Plaid transactions API every hour wastes API calls for users with no new transactions. Register a webhook URL with Plaid and receive push notifications when new transactions arrive, accounts are updated, or connection issues occur. This reduces API calls by 90% while improving data freshness.

Why It Matters

Plaid bank integration is the data layer that makes personal finance applications actually useful. Without it, users must manually enter every transaction. With it, the application has complete, real-time visibility into income and expenses without any user effort beyond the initial connection. The encrypted token storage

pattern shown here protects sensitive credentials using the same field-level encryption used by production fintech applications.

Chapter 24

Affordability Assessment Engine

14-Day Bill Forecasting and Cash Flow Analysis

Project: bills-tracker-v2

Overview

Knowing what you owe is only half the picture. Knowing whether you can afford it, given your current balance and upcoming income, is what prevents overdrafts and financial stress. The Affordability Assessment Engine in bills-tracker-v2 projects the user's financial position 14 days forward by combining current bank balance from Plaid, scheduled recurring bills, expected income deposits, and irregular spending patterns derived from transaction history.

The engine computes a daily cash flow projection that shows balance trajectory over the next 14 days. It identifies specific days where the projected balance falls below a configurable safety threshold, computes the deficit, and generates prioritized payment recommendations. Bills that cannot be paid with current funds are flagged with alternative strategies (payment plan, deferral, or minimum payment).

How It Works

- Current balance is fetched from Plaid accounts API and used as the starting point
- Scheduled bills are ordered by due date and deducted from the projected balance on their due dates
- Expected income deposits (regular paycheck, side income) are added based on historical deposit patterns
- Average daily discretionary spending is computed from the last 30 days of categorized transactions
- Balance is projected day by day for 14 days incorporating all known inflows and outflows
- Days where projected balance falls below the safety buffer generate urgency-ranked alerts
- Payment recommendations prioritize by due date, late fee cost, and credit impact

forecasting/affordability.py - 14-day cash flow projection engine

```python
async def assess_affordability(user_id: str) -> AffordabilityReport:
    balance = await get_current_balance(user_id)
    bills = await get_upcoming_bills(user_id, days=14)
    income = await get_expected_income(user_id, days=14)
    daily_spend = await get_avg_daily_spending(user_id)
    SAFETY_BUFFER = 200.0

    projected = balance
    timeline = []
    for day in range(14):
        date = date.today() + timedelta(days=day)
        projected -= daily_spend
        due_today = [b for b in bills if b.due_date == date]
        projected -= sum(b.amount for b in due_today)
        income_today = [i for i in income if i.expected_date == date]
        projected += sum(i.amount for i in income_today)
        timeline.append(DayProjection(
            date=date, balance=projected,
            is_critical=projected < SAFETY_BUFFER,
            bills_due=due_today,
        ))
    return AffordabilityReport(
        current_balance=balance,
        projected_minimum=min(d.balance for d in timeline),
        critical_days=[d for d in timeline if d.is_critical],
        timeline=timeline,
    )
```

■■ Include a Safety Buffer in All Projections

A projected balance of $1.00 on day 12 is not safe - a single unexpected expense or delayed deposit causes an overdraft. Include a configurable safety buffer (default $200) as a minimum balance threshold. Any day projected below this buffer is flagged as critical even if technically non-negative.

■ Base Daily Spend on 30-Day Rolling Average

Daily spending varies enormously. Rather than using last week's spending (which may be atypically high or low), use a 30-day rolling average as the daily spend baseline. This smooths out weekend shopping trips, monthly subscriptions, and other irregular patterns that would make 7-day averages unreliable.

Why It Matters

Overdraft fees cost Americans $8-15 billion annually. Most of these are predictable and preventable with a few days of advance warning. The 14-day affordability assessment provides that warning automatically, giving users time to adjust spending, delay non-critical purchases, or move money from savings before an overdraft occurs. This single feature turns bills-tracker-v2 from a tracking tool into an active financial guardian.

Chapter 25

Bill Anomaly Detection

Spike, Duplicate, and Cancellation Detection

Project: bills-tracker-v2

Overview

Recurring bills should be predictable. A water bill that has been $45 every month for two years should not suddenly be $312. A streaming subscription that was cancelled should stop generating charges. Duplicate charges from double-billing errors should be caught immediately. The bills-tracker-v2 Bill Anomaly Detection system monitors every incoming transaction against historical patterns and immediately alerts users when an anomaly is detected.

The detection system uses three distinct anomaly detectors that run in parallel on each new transaction: a spike detector that flags charges significantly above the historical mean, a duplicate detector that identifies charges with identical amount, merchant, and timing patterns, and a cancellation detector that flags charges from merchants where cancellation was previously recorded. Each detector produces a confidence score that is combined into a final anomaly report.

How It Works

- Each new transaction triggers the anomaly detection pipeline automatically on insertion
- The spike detector computes z-score relative to the last 12 months of charges from the same merchant
- A z-score above 2.0 (2 standard deviations) triggers a spike alert with severity proportional to the z-score
- The duplicate detector checks for identical (amount, merchant) pairs within a 48-hour window
- The cancellation detector cross-references merchant names against a cancellation log with fuzzy matching
- Anomaly confidence scores above 0.8 trigger immediate Telegram notifications with transaction details
- False positive feedback from users is used to update thresholds and improve detection accuracy

anomaly/detector.py - Z-score based spike detection with confidence scoring

```python
from scipy import stats

class BillAnomalyDetector:
    async def detect_spike(self, txn: Transaction) -> AnomalyResult:
        history = await self.db.fetchall(
            "SELECT amount FROM transactions WHERE merchant=? AND months_ago <= 12",
            (txn.merchant,)
        )
        if len(history) < 3:
            return AnomalyResult(detected=False)
        amounts = [r["amount"] for r in history]
        mean, std = stats.mean(amounts), stats.stdev(amounts)
        z_score = (txn.amount - mean) / (std + 1e-9)
        if z_score > 2.0:
            return AnomalyResult(
                detected=True, type="spike",
                confidence=min(1.0, (z_score - 2.0) / 3.0 + 0.6),
                message=(
                    f"{txn.merchant} charged ${txn.amount:.2f}"
                    f" (usual: ${mean:.2f}, {z_score:.1f}x normal)"
                ),
            )
        return AnomalyResult(detected=False)
```

■ Z-Score Threshold of 2.0 Balances Sensitivity

A z-score threshold below 2.0 generates too many false positives for normal price fluctuations. A threshold above 3.0 misses genuine spikes until they are extreme. Two standard deviations catches roughly the top 2.5% of unusual charges - a reasonable balance between sensitivity and false positive rate for billing anomalies.

■ Fuzzy Match for Merchant Name Variations

The same merchant may appear as "NETFLIX.COM", "NETFLIX INC", or "NETFLIX*PLAN" in different transactions. Use fuzzy string matching (difflib.SequenceMatcher or rapidfuzz) with a similarity threshold of 0.8 for cancellation and duplicate detection rather than exact string matching.

Why It Matters

Billing anomalies cost consumers billions annually through uncaught duplicate charges, undisclosed price increases, and charges from cancelled services. The anomaly detection system catches these issues within hours of occurrence rather than weeks later when the user manually reviews their bank statement. With automated detection and immediate notifications, users can dispute charges while they are still within the easy-dispute window.

PART 6

DeFi & Blockchain

Chapter 26

EIP712 Order Signing for Polymarket CLOB

Cryptographic Order Authentication on Polygon

Project: polymarket-bot

Overview

Decentralized prediction markets like Polymarket use a Central Limit Order Book (CLOB) architecture where orders are signed cryptographically by the trader's wallet. EIP712 is the Ethereum standard for structured data signing that makes these signatures human-readable in wallet interfaces. This chapter documents the EIP712 signing pattern as a cryptography and blockchain engineering study — not as a guide to trading or financial activity.

The signing process involves constructing a structured order object that matches Polymarket's CLOB contract schema, computing the EIP712 typed data hash, signing it with the private key using eth_account, and including the signature in the order submission API call. The signed order proves to the CLOB smart contract that the order was authorized by the wallet owner without requiring a blockchain transaction for every order placement.

How It Works

- EIP712 defines a standard for signing structured data with typed fields and a domain separator
- The domain separator uniquely identifies the Polymarket CLOB contract on Polygon mainnet
- The order type definition specifies fields: maker, taker, tokenId, makerAmount, takerAmount, expiration
- eth_account.sign_typed_data() computes the EIP712 hash and signs it with the private key
- The resulting v, r, s signature components are encoded and included in the order submission
- The CLOB smart contract verifies the signature matches the maker address before accepting the order
- Order signatures expire after a configurable time to prevent stale orders from being executed

signing/eip712.py - EIP712 order signing for Polymarket CLOB

```python
from eth_account import Account
from eth_account.structured_data import hash_domain

DOMAIN = {
    "name": "Polymarket CTF Exchange",
    "version": "1",
    "chainId": 137,  # Polygon
    "verifyingContract": CLOB_CONTRACT_ADDRESS,
}

ORDER_TYPE = {
    "Order": [
        {"name": "maker",        "type": "address"},
        {"name": "taker",        "type": "address"},
        {"name": "tokenId",      "type": "uint256"},
        {"name": "makerAmount",  "type": "uint256"},
        {"name": "takerAmount",  "type": "uint256"},
        {"name": "expiration",   "type": "uint256"},
        {"name": "nonce",        "type": "uint256"},
        {"name": "feeRateBps",   "type": "uint256"},
        {"name": "side",         "type": "uint8"},
    ]
}

def sign_order(order: dict, private_key: str) -> str:
    typed_data = {
        "types": ORDER_TYPE,
        "domain": DOMAIN,
        "primaryType": "Order",
        "message": order,
    }
    signed = Account.sign_typed_data(private_key, typed_data)
    return signed.signature.hex()
```

■ EIP712 Makes Signatures Human-Readable

Without EIP712, wallet signing dialogs show opaque hex bytes that users cannot verify. EIP712 typed data signing shows the structured fields - maker address, token ID, amounts, expiration - in a human-readable format so users can verify what they are signing before approving. This is critical for trust in decentralized applications.

■■ Never Expose Private Keys

The private_key parameter must come from a secure key management system - environment variables encrypted at rest, AWS Secrets Manager, or a hardware wallet. Never hardcode private keys in source code, log them, or include them in error messages. A compromised private key means total loss of all funds.

Why It Matters

EIP712 signing is the authentication layer that makes decentralized trading possible without requiring a blockchain transaction (and gas fee) for every order. Understanding the signing flow is essential for building any bot that interacts with decentralized exchanges, NFT marketplaces, or DeFi protocols. The same pattern applies to OpenSea orders, Uniswap permits, and any protocol that uses EIP712 signed messages for authorization.

Chapter 27

Web3 Wallet Management on Polygon

Transaction Signing, Gas Estimation, and Nonce Management

Project: polymarket-bot

Overview

Programmatic interaction with blockchain smart contracts requires managing several low-level concerns that do not exist in traditional web development: transaction nonce sequencing, gas price estimation, ERC-20 token approval workflows, and handling transaction confirmation with receipt validation. The polymarket-bot Web3 wallet management layer abstracts these concerns into a clean API that the trading strategy layer uses without needing to understand Polygon's transaction model.

The wallet manager handles the complete transaction lifecycle: estimating gas for each transaction type, fetching the current nonce and incrementing it locally to avoid sequential transaction delays, signing transactions with the private key, submitting to Polygon's RPC endpoint, and waiting for on-chain confirmation with configurable timeout and retry logic. All USDC operations include the ERC-20 approve step before the main transaction.

How It Works

- The Web3 provider connects to Polygon via a JSON-RPC endpoint (Alchemy, Infura, or public RPC)
- Nonce is fetched from the network on startup and incremented locally for sequential transactions
- Gas price uses eth_gasPrice with a configurable multiplier for faster inclusion priority
- ERC-20 token approval is checked before any trade to ensure the CLOB contract can spend USDC
- Transactions are signed locally with Account.sign_transaction() before broadcasting
- web3.eth.wait_for_transaction_receipt() polls for confirmation with configurable timeout
- Failed transactions (gas too low, nonce mismatch) trigger automatic retry with adjusted parameters

wallet/polygon.py - Complete transaction lifecycle management on Polygon

```python
from web3 import Web3
from eth_account import Account

class PolygonWallet:
    def __init__(self, private_key: str, rpc_url: str):
        self.w3 = Web3(Web3.HTTPProvider(rpc_url))
        self.account = Account.from_key(private_key)
        self.address = self.account.address
        self._nonce = self.w3.eth.get_transaction_count(self.address)

    def send_transaction(self, to: str, data: bytes,
                         value: int = 0) -> str:
        gas_price = int(self.w3.eth.gas_price * 1.2)  # 20% priority premium
        tx = {
            "from": self.address,
            "to": to,
            "value": value,
            "data": data,
            "nonce": self._nonce,
            "gas": 150_000,  # estimated upper bound
            "gasPrice": gas_price,
            "chainId": 137,  # Polygon
        }
        signed = self.account.sign_transaction(tx)
        tx_hash = self.w3.eth.send_raw_transaction(signed.rawTransaction)
        self._nonce += 1  # increment locally for next tx
        receipt = self.w3.eth.wait_for_transaction_receipt(tx_hash, timeout=60)
        if receipt.status != 1:
            raise ValueError(f"Transaction failed: {tx_hash.hex()}")
        return tx_hash.hex()
```

■ Local Nonce Management Enables Transaction Batching

Fetching the nonce from the network before every transaction causes sequential delays because each transaction must confirm before the next can be submitted. Managing nonce locally allows you to queue and broadcast multiple transactions immediately, letting the network sequence them by nonce without waiting for confirmations.

■■ Always Validate Transaction Receipt Status

A transaction that was included in a block can still fail - the EVM sets status=0 if the smart contract reverted. Always check receipt.status == 1 after wait_for_transaction_receipt(). A status=0 receipt means your funds moved but the operation did not execute, which requires specific recovery steps.

Why It Matters

Web3 wallet management is the plumbing layer that all on-chain trading bots require. Getting the nonce management, gas estimation, and receipt validation right is the difference between a bot that executes reliably and one that jams with stuck transactions or silently loses money on failed transactions. This implementation handles all the edge cases that simpler examples omit, making it suitable for production trading bot deployment.

Chapter 28

Flash Loan Arbitrage Detection

Cross-DEX Price Discrepancy with Exponential Backoff

Project: flash-arb

Overview

Flash loans allow DeFi traders to borrow unlimited capital within a single transaction without collateral, execute arbitrage across decentralized exchanges, and repay the loan plus fee in the same transaction - all atomically. If the arbitrage fails, the entire transaction reverts and nothing happens. The flash-arb project monitors price discrepancies between Uniswap V3 and SushiSwap using the DEX subgraph APIs, identifying opportunities where the same token pair trades at different prices on different platforms.

Profitable flash loan arbitrage opportunities last for seconds before other arbitrageurs close the price gap. The detection system runs continuously, querying both DEX subgraphs for current pool prices, computing the theoretical arbitrage profit after accounting for flash loan fees (0.09%), DEX swap fees, and gas costs. Only opportunities with positive net profit after all costs are flagged for execution.

How It Works

- Subgraph queries fetch current token0Price and token1Price for target pools on both DEXes
- Price discrepancy is computed as percentage difference between Uniswap and SushiSwap prices
- Flash loan fee (0.09%), DEX swap fees (0.3% each), and estimated gas cost are deducted from gross profit
- Net profit must exceed a minimum threshold (0.5%) to justify execution risk
- Exponential backoff on subgraph API failures prevents rate limiting during outages
- Detected opportunities are logged with expected profit, optimal trade size, and execution window
- The actual flash loan execution uses an Aave V3 flash loan contract on Polygon

detection/arb_scanner.py - Cross-DEX price discrepancy detection

```python
import asyncio
import httpx

UNISWAP_SUBGRAPH = "https://api.thegraph.com/subgraphs/name/uniswap/uniswap-v3"
SUSHI_SUBGRAPH   = "https://api.thegraph.com/subgraphs/name/sushiswap/exchange"

PRICE_QUERY = """
{
  pool(id: "%s") {
    token0Price
    token1Price
    liquidity
  }
}
"""

async def detect_arb(
    pool_id_uni: str, pool_id_sushi: str, min_profit_pct: float = 0.005
) -> ArbitrageOpportunity | None:
    async with httpx.AsyncClient() as client:
        uni, sushi = await asyncio.gather(
            client.post(UNISWAP_SUBGRAPH,
                        json={"query": PRICE_QUERY % pool_id_uni}),
            client.post(SUSHI_SUBGRAPH,
                        json={"query": PRICE_QUERY % pool_id_sushi})
        )
    price_uni   = float(uni.json()["data"]["pool"]["token0Price"])
    price_sushi = float(sushi.json()["data"]["pool"]["token0Price"])
    spread = abs(price_uni - price_sushi) / min(price_uni, price_sushi)
    # Account for fees: 0.09% flash loan + 0.3%+0.3% swap fees
    net_profit_pct = spread - 0.0009 - 0.003 - 0.003
    if net_profit_pct >= min_profit_pct:
        return ArbitrageOpportunity(
            buy_on="uniswap" if price_uni < price_sushi else "sushiswap",
            sell_on="sushiswap" if price_uni < price_sushi else "uniswap",
            net_profit_pct=net_profit_pct,
        )
    return None
```

■■ MEV Bots Make Most Arb Unprofitable

On mainnet Ethereum, Maximum Extractable Value (MEV) bots with direct validator connections execute arbitrage in the same block as the price discrepancy appears. Most retail-accessible arbitrage opportunities on Ethereum mainnet are not profitable. On Polygon and other L2s with lower competition, smaller opportunities remain viable.

■ asyncio.gather() for Parallel Price Fetching

Fetching prices from two subgraphs sequentially doubles the latency and increases the chance that prices change between the two fetches, making the price comparison stale. asyncio.gather() fetches both simultaneously, minimizing the window for price movement between observations.

Why It Matters

Flash loan arbitrage is one of the few strategies where a retail trader can access the same capital depth as institutional players. The atomicity guarantee means zero risk of losing the borrowed capital - if the trade is not profitable, it simply reverts. Understanding the detection and execution pattern is valuable even for non-DeFi applications: the same parallel price comparison architecture applies to cross-exchange arbitrage on centralized exchanges.

PART 7

Content & Media Pipelines

Chapter 29

ElevenLabs TTS with Character Voice Mapping

Multi-Voice Text-to-Speech with Fallback Chain

Project: pricedin-shorts

Overview

The pricedin-shorts project generates short-form financial explainer videos where different "characters" - a dollar bill explaining inflation, a credit card discussing interest rates, a stock chart narrating market moves - each have distinct personalities and voices. ElevenLabs provides the TTS (text-to-speech) API, and the Voice Mapping system assigns specific ElevenLabs voice IDs to each character, with a configurable fallback chain for when specific voices are unavailable.

The voice generation pipeline handles the complete audio production workflow: mapping character IDs to ElevenLabs voice IDs, constructing API requests with character-appropriate voice settings (stability and similarity boost), managing API key rotation across multiple keys to handle rate limits, and caching generated audio segments to avoid regenerating identical text passages in revised scripts.

How It Works

- VOICE_MAP dictionary maps character ID strings to specific ElevenLabs voice ID hashes
- Voice settings (stability, similarity_boost, style) are tuned per character for consistent personality
- The model eleven_multilingual_v2 supports natural-sounding speech in multiple languages
- API key rotation cycles through multiple keys when the primary key hits its monthly character limit
- Generated audio segments are cached by text hash to enable script revision without re-generating unchanged lines
- A fallback chain tries character-specific voice, then category voice, then default voice on API errors
- Returned audio bytes are saved as MP3 files named by segment order for video assembly

audio/elevenlabs.py - Character voice mapping with API key rotation fallback

```python
VOICE_MAP = {
    "dollar_bill":  "TxGEqnHWrfWFTfGW9XjX",
    "credit_card":  "AZnzlk1XvdvUeBnXmlld",
    "stock_chart":  "21m00Tcm4TlvDq8ikWAM",
    "narrator":     "pNInz6obpgDQGcFmaJgB",
}
DEFAULT_VOICE = "pNInz6obpgDQGcFmaJgB"

def generate_voice(character_id: str, text: str,
                   stability: float = 0.5) -> bytes:
    voice_id = VOICE_MAP.get(character_id, DEFAULT_VOICE)
    for api_key in API_KEYS:  # key rotation
        try:
            r = requests.post(
                f"https://api.elevenlabs.io/v1/text-to-speech/{voice_id}",
                headers={
                    "xi-api-key": api_key,
                    "Content-Type": "application/json",
                },
                json={
                    "text": text,
                    "model_id": "eleven_multilingual_v2",
                    "voice_settings": {
                        "stability": stability,
                        "similarity_boost": 0.75,
                        "style": 0.0,
                    },
                }
            )
            r.raise_for_status()
            return r.content
        except requests.HTTPError as e:
            if e.response.status_code == 429:
                continue  # try next key
            raise
    raise RuntimeError("All API keys exhausted")
```

■ Stability Controls Voice Consistency

stability=0.5 balances between expressive variation and consistency. For narrators and explainers, use 0.7+ for predictable, authoritative delivery. For character voices with personality, use 0.3-0.5 for more expressive, engaging speech. Lower stability increases variation between regenerations of the same text.

■ Cache Audio by Text Hash

Generating a 30-second audio segment costs 150-200 ElevenLabs characters and takes 3-5 seconds. When revising a script, cache previously generated segments by SHA256 hash of the text and character ID. This saves both API credits and generation time when only a few lines change.

Why It Matters

High-quality TTS with distinct character voices transforms text-based content into engaging audio-visual experiences. The voice mapping system makes the pricedin-shorts characters recognizable and consistent across hundreds of videos - viewers come to associate the dollar_bill voice with inflation discussions and the stock_chart voice with market analysis. This consistency is what builds audience familiarity and channel identity at scale.

Chapter 30

Frame-by-Frame Video Animation Engine

Programmatic Video Generation with PIL

Project: pricedin-shorts

Overview

Creating animated explainer videos programmatically requires compositing multiple visual layers - background, character sprites, text overlays, progress indicators - into individual frames, then assembling those frames into a video at the correct frame rate. The pricedin-shorts animation engine uses Python Imaging Library (PIL/Pillow) for frame composition and FFmpeg for video assembly, generating 60-90 second animated videos entirely in code without any design tool involvement.

Each video consists of scenes, each scene consists of frames, and each frame is composited by layering: background gradient, character sprite at computed position, subtitle text with outline for readability, progress bar at the bottom, and any data visualization elements (charts, numbers, percentage indicators). The system generates 24 frames per second, so a 60-second video requires 1,440 individual frame composites.

How It Works

- Scene definitions specify character, duration, text, and animation type (slide-in, static, zoom)
- Each frame is composited as a 1080x1920 pixel image (vertical format for Shorts/Reels/TikTok)
- PIL Image.new() creates the base frame with the background color or gradient
- Character sprites are loaded, resized, and pasted at position computed by animation easing functions
- Text is rendered with PIL ImageDraw with custom fonts, sizing, and drop shadow for readability
- Completed frames are saved as sequential JPEG files named frame_00001.jpg through frame_n
- FFmpeg assembles frames into an MP4 with the matching audio track from ElevenLabs TTS

video/renderer.py - PIL frame composition with FFmpeg video assembly

```python
from PIL import Image, ImageDraw, ImageFont
import subprocess

def render_frame(scene: Scene, frame_idx: int,
                 audio_offset: float) -> Image.Image:
    img = Image.new("RGB", (1080, 1920), color="#0D1117")
    draw = ImageDraw.Draw(img)

    # Background gradient
    for y in range(1920):
        r = int(13 + (y / 1920) * 10)
        img.putpixel((0, y), (r, 17, 23))

    # Character sprite
    sprite = Image.open(f"assets/{scene.character}.png").convert("RGBA")
    sprite = sprite.resize((400, 400))
    t = frame_idx / scene.fps
    y_pos = 600 - int(10 * math.sin(t * 2 * math.pi))
    img.paste(sprite, (340, y_pos), sprite)

    # Subtitle text with outline
    font = ImageFont.truetype("assets/Inter-Bold.ttf", 52)
    text = scene.get_text_at(frame_idx)
    draw.text((540, 1500), text, font=font, fill="white",
              stroke_width=3, stroke_fill="black", anchor="mm")

    return img

def assemble_video(frames_dir: str, audio_path: str, out_path: str):
    subprocess.run([
        "ffmpeg", "-framerate", "24",
        "-i", f"{frames_dir}/frame_%05d.jpg",
        "-i", audio_path,
        "-c:v", "libx264", "-c:a", "aac",
        "-shortest", out_path
    ], check=True)
```

■ Easing Functions Make Animation Smooth

Linear interpolation between positions looks robotic. Use easing functions (ease-in-out, elastic, bounce) from the pytweening library to make character movements feel natural. The difference between linear and ease-in-out for a 30-frame character slide-in is immediately noticeable and makes videos look professionally produced.

■ JPEG Quality 85 Balances Size and Quality

Saving frames as JPEG with quality=85 produces files 3-4x smaller than PNG with imperceptible quality loss at 1080p resolution. This matters when generating 1,440+ frames - PNG frames can exceed 10GB per video while JPEG stays under 2GB. FFmpeg handles JPEG frame input natively.

Why It Matters

Programmatic video generation at scale is impossible with manual design tools. The pricedin-shorts engine generates 7 videos per week with consistent visual quality, brand guidelines, and character animation - a volume that would require a full design team if done manually. The same PIL-plus-FFmpeg pattern applies to any domain requiring automated video content: educational explainers, data visualization videos, product demo animations, and news digest videos.

Chapter 31

AI Script Generation with Groq

Modular Format System for Video Scripts

Project: pricedin-shorts

Overview

High-quality short-form financial videos require scripts that are simultaneously informative, engaging, concise, and structured for visual animation. Generating these scripts manually for 7 videos per week is unsustainable. The pricedin-shorts script generation system uses Groq's ultra-fast inference API (powered by LPU hardware) to generate complete video scripts in under 2 seconds, with a modular format system that ensures each script conforms to the animation engine's scene structure requirements.

The script generation pipeline takes a financial topic or news event, classifies it into one of several content formats (explainer, news reaction, comparison, prediction), selects the appropriate character for the topic, and generates a structured JSON script with scenes, character assignments, spoken text, and visual cues. The structured output is validated against the ScriptSchema before being passed to the animation engine.

How It Works

- Content formats define templates: explainer (concept + examples + impact), news (event + context + reaction)
- Topic classification determines which format and character combination to use
- The Groq API with llama-3.1-70b provides fast, high-quality script generation at low cost
- Temperature 0.7 balances creativity with consistency in script tone and structure
- The system prompt includes the complete character personalities and voice guidelines
- Structured JSON output is validated against ScriptSchema using Pydantic before acceptance
- Failed validations trigger a retry with the error message as correction context (from Chapter 3)

content/script_gen.py - Groq-powered structured script generation with Pydantic

```python
from groq import Groq
from pydantic import BaseModel

class Scene(BaseModel):
    character: str
    text: str
    duration_seconds: float
    animation: str  # "static" | "slide_in" | "zoom"

class Script(BaseModel):
    title: str
    topic: str
    scenes: list[Scene]
    total_duration: float

SCRIPT_PROMPT = """
Create a {format_type} video script about: {topic}

Available characters: dollar_bill, credit_card, stock_chart, narrator

Return JSON matching this schema:
{{"title": str, "topic": str, "scenes": [
  {{"character": str, "text": str, "duration_seconds": float, "animation": str}}
], "total_duration": float}}

Constraints:
- Total duration: 50-70 seconds
- Each scene: 8-15 seconds of text
- Start with narrator, use topic-specific character for main content
"""

def generate_script(topic: str, format_type: str = "explainer") -> Script:
    client = Groq(api_key=GROQ_API_KEY)
    response = client.chat.completions.create(
        model="llama-3.1-70b-versatile",
        messages=[{"role": "user",
                   "content": SCRIPT_PROMPT.format(
                       topic=topic, format_type=format_type)}],
        temperature=0.7,
        response_format={"type": "json_object"},
    )
    return Script.model_validate_json(response.choices[0].message.content)
```

■ Groq LPU Speed Is a Game Changer

Groq's Language Processing Units (LPU) deliver 500+ tokens per second versus GPT-4's 30-50 tokens per second. For script generation, this means complete scripts in 1-2 seconds instead of 15-30 seconds. For content pipelines processing hundreds of topics per day, this throughput difference is transformative.

■ Structured Output Eliminates Parsing Failures

Asking a model to "return JSON" without using the JSON response format option leads to frequent failures: markdown code blocks, trailing commas, comments in JSON, and truncated objects. Using response_format=json_object forces valid JSON output and eliminates an entire category of parsing errors.

Why It Matters

AI script generation transforms a content creation bottleneck into a fully automated pipeline. The modular format system ensures that generated scripts always conform to the animation engine's requirements, eliminating the manual review step that would otherwise be needed for every AI-generated script. The Groq integration demonstrates the practical impact of inference speed on pipeline throughput - a 10x speed improvement in the generation step translates to dramatically higher daily content volume.

Chapter 32

Content Deduplication Brain

SQLite Sequel Detection and Topic Clustering

Project: pricedin-shorts

Overview

A daily-publishing financial video channel that covers the same topic twice in a week loses viewer trust and algorithmic favor with platforms. The pricedin-shorts Content Deduplication Brain prevents this by maintaining a SQLite database of all published topics with their semantic embeddings, and checking each new topic candidate against this history before generating a script. Similar topics (like "inflation explained" and "what causes inflation") are detected even when their titles differ significantly.

The deduplication system uses sentence-transformers to generate semantic embeddings of each topic description, stores them in the database with a BLOB column, and computes cosine similarity between any new candidate topic and all published topics from the last 30 days. Topics with similarity above 0.85 are flagged as potential sequels or duplicates, prompting the content scheduler to select a different topic.

How It Works

- Sentence-transformers generates a 384-dimensional embedding vector for each topic description
- Embeddings are stored as BLOB in SQLite alongside topic title, publish date, and video ID
- New topic candidates are embedded and cosine similarity is computed against all recent topics
- A similarity threshold of 0.85 identifies near-duplicate topics that differ in phrasing but not substance
- Clusters of related topics help the scheduler maintain topical diversity across the content calendar
- Topics below the similarity threshold are cleared for production and added to the database on publish
- The deduplication window is configurable: default 30 days, extended to 90 for evergreen content

dedup/brain.py - Semantic deduplication with sentence-transformers and SQLite

```python
import sqlite3
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

def embed_topic(topic: str) -> bytes:
    vec = model.encode(topic, normalize_embeddings=True)
    return vec.astype(np.float32).tobytes()

def cosine_similarity(a: bytes, b: bytes) -> float:
    va = np.frombuffer(a, dtype=np.float32)
    vb = np.frombuffer(b, dtype=np.float32)
    return float(np.dot(va, vb))  # pre-normalized vectors

def is_duplicate(topic: str, db: sqlite3.Connection,
                 threshold: float = 0.85) -> tuple[bool, str | None]:
    new_emb = embed_topic(topic)
    rows = db.execute(
        "SELECT title, embedding FROM topics"
        " WHERE published_at > date('now', '-30 days')"
    ).fetchall()
    for title, stored_emb in rows:
        similarity = cosine_similarity(new_emb, stored_emb)
        if similarity >= threshold:
            return True, f"Too similar to recent: {title} ({similarity:.2f})"
    return False, None
```

■ Normalize Embeddings Before Storage

Storing L2-normalized embeddings means cosine similarity reduces to a simple dot product - much faster than computing full cosine similarity with division. Pre-normalize with normalize_embeddings=True in SentenceTransformer.encode() and store as float32 bytes for compact, fast similarity computation.

■ Tune Threshold for Your Content Domain

A similarity threshold of 0.85 works well for financial topics where different phrasing often means the same concept. For more diverse content categories, use 0.90 to allow more variation. Test your threshold against known duplicate pairs from your content history before deploying.

Why It Matters

Content deduplication at scale is a solved problem in search engines but rarely implemented in content creation pipelines. The semantic embedding approach catches duplicates that keyword-based approaches miss entirely - "what is compound interest" and "how compound interest grows your money" would not match on keywords but score 0.91 similarity with sentence-transformers. This intelligence is what maintains content quality and diversity at the publishing volumes that AI-assisted creation enables.

Chapter 33

YouTube Data API v3 - Automated Video Upload

Programmatic Video Publishing with Metadata

Project: pricedin-shorts

Overview

The final step in the pricedin-shorts content pipeline is uploading the generated video to YouTube with optimized metadata. The YouTube Data API v3 provides a resumable upload protocol that handles large files reliably, with automatic retry of failed chunks. The upload pipeline sets title, description, tags, category, and schedule - enabling fully automated publishing without manual YouTube Studio interaction.

Beyond the upload itself, the system handles thumbnail generation and upload, playlist management, and chapter markers in the description. The YouTube API requires OAuth2 authentication rather than API keys for upload operations, so the system uses a service account or OAuth2 credentials with refresh token storage to maintain long-term access without requiring periodic re-authentication.

How It Works

- OAuth2 credentials with offline access scope are obtained once and stored with refresh token
- The google-auth library automatically refreshes expired access tokens using the stored refresh token
- Resumable upload protocol handles video files in chunks with automatic retry on network interruption
- Video metadata includes title, description, tags, categoryId, defaultLanguage, and privacyStatus
- Scheduled publishing uses publishAt with RFC 3339 timestamp for future publication time
- Thumbnail upload requires a separate API call after the video upload completes
- Playlist insertion adds the video to the channel's Shorts playlist automatically

publishing/youtube.py - Resumable video upload with thumbnail and metadata

```python
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload
from google.oauth2.credentials import Credentials

def upload_video(
    video_path: str, thumbnail_path: str, metadata: VideoMetadata
) -> str:
    creds = Credentials.from_authorized_user_file("token.json")
    youtube = build("youtube", "v3", credentials=creds)

    body = {
        "snippet": {
            "title": metadata.title,
            "description": metadata.description,
            "tags": metadata.tags,
            "categoryId": "22",  # People & Blogs
            "defaultLanguage": "en",
        },
        "status": {
            "privacyStatus": "private",  # change to public when ready
            "selfDeclaredMadeForKids": False,
        }
    }
    media = MediaFileUpload(video_path, chunksize=1024*1024,
                            resumable=True, mimetype="video/mp4")
    request = youtube.videos().insert(
        part="snippet,status", body=body, media_body=media
    )
    response = None
    while response is None:
        _, response = request.next_chunk()
    video_id = response["id"]
    # Upload thumbnail
    youtube.thumbnails().set(
        videoId=video_id,
        media_body=MediaFileUpload(thumbnail_path)
    ).execute()
    return video_id
```

■ Upload as Private First

Always upload videos as privacyStatus: private and switch to public after verifying the upload completed correctly, thumbnail applied, and metadata looks correct. Uploading directly as public means any errors in metadata or thumbnail will be visible to subscribers during the correction window.

■■ Resumable Upload Is Required for Large Files

The simple upload method fails for files over 5MB and is unreliable for files over 50MB. Always use MediaFileUpload with resumable=True for video files. Resumable uploads automatically retry failed chunks and can resume from interruption, which is critical for 50-500MB video files on unreliable connections.

Why It Matters

Automated video publishing is what transforms a content generation system into a content machine. Without automated upload, every AI-generated video still requires manual YouTube Studio interaction - title input, description, tags, thumbnail, scheduling. Automating this final step with the YouTube Data API v3 completes the pipeline from topic selection to published video with zero manual intervention, enabling sustainable high-frequency publishing.

Chapter 34

Telegram Review Gate

Human-in-the-Loop Approval for AI Content

Project: pricedin-shorts

Overview

Fully automated content pipelines risk publishing errors that could damage brand reputation: factual mistakes, inappropriate tone, incorrect financial data, or poor video quality. The pricedin-shorts Telegram Review Gate inserts a human approval step between video generation and publication without blocking the pipeline. The system sends the generated video and metadata to a Telegram chat for review, awaits approval or rejection via inline keyboard buttons, and proceeds with publication only on explicit approval.

The review gate pattern is the practical implementation of "human-in-the-loop" AI systems - automation handles the high-volume, repetitive work, while humans review and approve at critical decision points. The Telegram interface makes review accessible from any mobile device without requiring access to a production environment. Rejection sends the video back through the pipeline with the reviewer's feedback notes.

How It Works

- The pipeline generates the video and stores it locally before sending for review
- telegram.send_video() sends the MP4 to a designated review channel with caption and metadata
- Inline keyboard with APPROVE and REJECT buttons is attached to the review message
- The bot listens for callback queries from the inline buttons with the video ID encoded in callback_data
- Approval triggers the YouTube upload pipeline with the stored video and metadata
- Rejection stores the reviewer's feedback note and re-queues the topic for regeneration
- A 24-hour timeout automatically rejects unreviewed videos to prevent pipeline stalls

review/telegram_gate.py - Human-in-the-loop approval with inline keyboard

```python
import telebot
from telebot.types import InlineKeyboardMarkup, InlineKeyboardButton

bot = telebot.TeleBot(TELEGRAM_TOKEN)
pending_reviews: dict[str, PendingVideo] = {}

def send_for_review(video: GeneratedVideo):
    markup = InlineKeyboardMarkup()
    markup.add(
        InlineKeyboardButton("Approve", callback_data=f"approve:{video.id}"),
        InlineKeyboardButton("Reject",  callback_data=f"reject:{video.id}"),
    )
    with open(video.path, "rb") as vf:
        msg = bot.send_video(
            REVIEW_CHAT_ID, vf,
            caption=f"*{video.title}*\n\nDuration: {video.duration}s",
            parse_mode="Markdown",
            reply_markup=markup,
        )
    pending_reviews[video.id] = video

@bot.callback_query_handler(lambda q: q.data.startswith("approve:"))
def handle_approve(query):
    video_id = query.data.split(":")[1]
    video = pending_reviews.pop(video_id)
    upload_to_youtube(video)  # triggers publication pipeline
    bot.answer_callback_query(query.id, "Publishing...")
```

■ Encode Video ID in Callback Data

callback_data is limited to 64 bytes. Encode only the video ID, not the full metadata. Store the full video object in a pending_reviews dict keyed by video ID and retrieve it when the callback fires. This keeps callback_data minimal while preserving all the information needed for processing.

■■ Always Set a Review Timeout

Without a timeout, a reviewer who goes on vacation leaves the pipeline permanently stalled. Set a 24-hour timeout that automatically rejects and re-queues unreviewed content. Send a reminder notification at the 12-hour mark for time-sensitive content. Never let the pipeline hang on human action indefinitely.

Why It Matters

The Telegram Review Gate demonstrates a practical and pragmatic approach to AI safety: rather than attempting to make the AI pipeline perfect (impossible), insert human judgment at the point where mistakes would be most costly (public publication). The pattern keeps the pipeline fast and high-throughput while ensuring that a human sees every piece of content before it reaches the audience. This is the correct architecture for any automated content system where quality and accuracy matter.

Chapter 35

TF-IDF News Story Clustering

Cosine Similarity for Deduplication and Grouping

Project: beacon

Overview

News aggregators receive hundreds of articles per hour covering the same stories from different sources. Without deduplication, a single major event generates 20+ near-identical articles that flood the feed and bury other stories. The beacon project implements TF-IDF vectorization with cosine similarity to cluster news articles by story, identify the canonical representative of each cluster, and deduplicate the aggregated feed while preserving unique coverage from different perspectives.

TF-IDF (Term Frequency-Inverse Document Frequency) represents each article as a sparse vector where high-weight terms are those that appear frequently in the article but rarely across all articles - giving high weight to distinctive terms while downweighting common words. Cosine similarity between these vectors measures thematic similarity. Articles with cosine similarity above 0.6 are grouped into the same story cluster.

How It Works

- Articles are collected from RSS feeds and stored with title, body, source, and publication timestamp
- TfidfVectorizer builds the term-document matrix from the current batch of articles
- cosine_similarity() computes pairwise similarity between all article pairs in the batch
- Agglomerative clustering groups articles above the similarity threshold into story clusters
- Each cluster's representative article is selected by recency and source authority ranking
- Unique articles (no cluster members above threshold) are kept as standalone stories
- Story clusters with 5+ articles indicate major breaking news warranting priority placement

clustering/tfidf_clusters.py - TF-IDF story clustering with cosine similarity

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def cluster_articles(
    articles: list[Article], similarity_threshold: float = 0.6
) -> list[StoryCluster]:
    corpus = [f"{a.title} {a.body[:500]}" for a in articles]
    vectorizer = TfidfVectorizer(
        max_features=5000,
        stop_words="english",
        ngram_range=(1, 2),  # unigrams and bigrams
    )
    tfidf_matrix = vectorizer.fit_transform(corpus)
    sim_matrix = cosine_similarity(tfidf_matrix)

    clusters: list[list[int]] = []
    assigned = set()
    for i, article in enumerate(articles):
        if i in assigned:
            continue
        similar = np.where(sim_matrix[i] >= similarity_threshold)[0].tolist()
        cluster = [j for j in similar if j not in assigned]
        clusters.append(cluster)
        assigned.update(cluster)

    return [
        StoryCluster(
            representative=articles[max(c, key=lambda j: articles[j].authority_score)],
            members=[articles[j] for j in c],
            size=len(c),
        )
        for c in clusters
    ]
```

■ Bigrams Capture Phrases TF-IDF Misses

Single-word TF-IDF misses the difference between "interest rate" and "rate of return" - both contain "rate". Setting ngram_range=(1,2) includes bigrams, allowing the model to assign high weight to distinctive phrases like "federal reserve" and "earnings per share" as units rather than individual words.

■ Threshold Tuning Matters for Your Content Domain

A threshold of 0.6 works for financial news where different sources report the same story with similar terminology. For more diverse content (technology, culture), use 0.7+ to avoid over-merging stories that share some topical vocabulary but are genuinely distinct. Evaluate on 100+ manually labeled pairs.

Why It Matters

News deduplication is a fundamental challenge for any news aggregation product. Without it, breaking news floods the entire feed with near-identical articles, burying other stories and degrading the user experience. TF-IDF clustering solves this at low computational cost with no GPU requirement, making it practical for real-time processing of hundreds of articles per hour on commodity hardware. The same clustering approach applies to support ticket deduplication, research paper grouping, and any domain requiring semantic document grouping.

PART 8

Security & Identity

Chapter 36

Fernet Field-Level Encryption Vault

Symmetric Encryption for Sensitive Data Storage

Project: identity-vault

Overview

Most cloud databases are encrypted at rest — disk encryption that protects data if the physical storage volume is stolen or decommissioned. This sounds reassuring until you consider what it does not cover: a compromised database password, a SQL injection vulnerability, a misconfigured backup bucket, or a rogue employee with database admin credentials. All of these scenarios result in plaintext data exposure because disk encryption is transparently unlocked by the running database process. Field-level encryption protects data before it ever reaches the database: the application encrypts each sensitive value using a master key that lives only in the application environment — an environment variable or secrets manager — never in the database itself. The database receives and stores ciphertext. Even with full database access, an attacker sees URL-safe Base64-encoded tokens that are meaningless without the master key stored elsewhere. This is the control that makes a database breach contained rather than catastrophic.

Fernet (from the Python cryptography library) provides authenticated encryption, and understanding what is inside each token reveals why it is genuinely secure. A Fernet token encodes five components in sequence: a version byte (0x80), an 8-byte timestamp in UNIX seconds, a 16-byte initialization vector (IV) chosen randomly per encryption call, the ciphertext encrypted with AES-128-CBC using the first 16 bytes of the 32-byte key, and a 32-byte HMAC-SHA256 authentication tag computed over all preceding bytes using the last 16 bytes of the key. The random IV ensures that encrypting the same value twice produces different ciphertexts — an attacker cannot identify duplicate values by comparing stored tokens. The HMAC ensures that any bit-flip in the stored ciphertext causes decryption to raise an InvalidToken exception rather than returning corrupted plaintext. The embedded timestamp enables key rotation by identifying when each value was encrypted.

How It Works

- The master key is stored in an environment variable or AWS Secrets Manager, never in source code
- Fernet.generate_key() generates a cryptographically secure 32-byte key encoded in URL-safe base64
- encrypt_text() converts a string value to encrypted bytes and returns URL-safe base64 ciphertext
- decrypt_text() reverses the process with automatic authentication verification
- Fernet tokens include a timestamp, enabling key rotation by re-encrypting values with the new key

- mask_value() reveals only the last 2-4 characters for display, replacing the rest with asterisks
- Encrypted values are stored as TEXT columns, making them compatible with all database systems

vault/cipher.py - Fernet field-level encryption with sensitivity-aware masking

```
from cryptography.fernet import Fernet
from enum import Enum

class SensitivityLevel(Enum):
    PRIVATE = "private"     # show last 2 chars
    SENSITIVE = "sensitive" # show last 4 chars

class VaultCipher:
    def __init__(self, master_key: str):
        self._fernet = Fernet(master_key.encode())

    def encrypt_text(self, value: str) -> str:
        return self._fernet.encrypt(value.encode()).decode()

    def decrypt_text(self, value: str) -> str:
        return self._fernet.decrypt(value.encode()).decode()

    def encrypt_json(self, data: dict) -> str:
        import json
        return self.encrypt_text(json.dumps(data))

    def decrypt_json(self, ciphertext: str) -> dict:
        import json
        return json.loads(self.decrypt_text(ciphertext))

def mask_value(value: str, sensitivity: SensitivityLevel) -> str:
    visible_tail = 2 if sensitivity == SensitivityLevel.PRIVATE else 4
    masked_len = max(len(value) - visible_tail, 0)
    return "*" * masked_len + value[-visible_tail:]
```

■■ Key Rotation Is a First-Class Concern

Store key version alongside encrypted values: fernet_v1:encrypted_data. When rotating to a new key, re-encrypt all values in a background job. Keep the old key active for decryption until all values are re-encrypted. Fernet tokens include timestamps that help identify when values were last encrypted.

■ Use MultiFernet for Zero-Downtime Key Rotation

cryptography.fernet.MultiFernet accepts a list of Fernet instances. Encryption uses the first key; decryption tries each key in order. This enables zero-downtime key rotation: add the new key first, run the re-encryption job, then remove the old key - all without any downtime.

■■ Common Pitfall: Encrypting Fields You Need to Query

Field-level encryption prevents the database from seeing plaintext values — which also means you cannot query encrypted fields with WHERE clauses or JOIN conditions. SELECT * FROM users WHERE email = ? becomes impossible when email is encrypted. Design your schema to store a searchable HMAC (keyed hash of the plaintext using a separate search key) alongside the encrypted ciphertext. Query by HMAC, fetch by primary key, decrypt on retrieval. Plan your queryability requirements before encrypting every field.

Why It Matters

Field-level encryption is the security control that determines whether a data breach is contained or catastrophic. Without it, a single SQL injection attack or compromised database credential exposes all user data in plaintext. With it, the attacker gets rows of ciphertext that are useless without the master key stored separately in the application environment. This is the standard security control required by HIPAA-compliant healthcare applications, PCI-DSS compliant payment systems, and any application handling data sensitive enough to cause real harm if exposed. Mastering this pattern means you can confidently build applications that handle medical records, financial data, and personal identity information with genuine security rather than hope.

Chapter 37

HMAC Constant-Time Comparison and SHA256 Hashing

Cryptographically Safe Signature Verification

Project: identity-vault

Overview

Webhook endpoints that receive callbacks from external services (Stripe, GitHub, Twilio) need to verify that the incoming request is genuinely from the expected sender and has not been tampered with. Simple string comparison of signatures is vulnerable to timing attacks where an attacker can determine the correct signature byte by byte by measuring response time differences. The identity-vault project implements HMAC-SHA256 signature verification using constant-time comparison to prevent timing attacks.

HMAC (Hash-based Message Authentication Code) uses a shared secret key combined with the request payload to generate a signature. The recipient independently computes the expected HMAC and compares it against the provided signature using hmac.compare_digest() which takes exactly the same time regardless of how many bytes match - preventing timing oracle attacks that could allow signature forgery.

How It Works

- The webhook sender computes HMAC-SHA256(secret, payload) and includes it in a header
- The receiver extracts the signature from the header and the raw payload bytes from the request body
- The receiver independently computes HMAC-SHA256(secret, payload) using the shared secret
- hmac.compare_digest() compares the expected and received signatures in constant time
- Regular string comparison (==) is vulnerable to timing attacks; compare_digest() is not
- The shared secret is stored in environment variables and never included in source code
- Timestamp validation prevents replay attacks by rejecting webhooks older than 5 minutes

security/hmac_verify.py - Constant-time HMAC verification and PBKDF2 hashing

```python
import hmac
import hashlib
import time

def compute_signature(payload: bytes, secret: str) -> str:
    return hmac.new(
        secret.encode(), payload, hashlib.sha256
    ).hexdigest()

def verify_webhook_signature(
    payload: bytes,
    signature_header: str,
    secret: str,
    timestamp_header: str | None = None
) -> bool:
    # Replay attack prevention
    if timestamp_header:
        ts = int(timestamp_header)
        if abs(time.time() - ts) > 300:  # 5 minute window
            return False
        # Include timestamp in signed payload (like Stripe)
        signed_payload = f"{ts}.".encode() + payload
    else:
        signed_payload = payload

    expected = compute_signature(signed_payload, secret)
    # CRITICAL: constant-time comparison prevents timing attacks
    return hmac.compare_digest(expected, signature_header)

def hash_password(password: str, salt: bytes | None = None) -> tuple[str, bytes]:
    import os
    salt = salt or os.urandom(32)
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100_000)
    return dk.hex(), salt
```

■■ Never Use == for Signature Comparison

Regular string equality (== or strcmp) returns as soon as it finds the first differing byte. An attacker can send thousands of signature guesses and measure response times to determine how many bytes match, eventually reconstructing the valid signature. hmac.compare_digest() always takes the same time regardless of where the mismatch occurs.

■ PBKDF2 with 100,000 Iterations for Passwords

Raw SHA256(password) is computed in microseconds - attackers can try billions of password candidates per second with GPU hardware. PBKDF2 with 100,000 iterations takes 100,000x longer, reducing GPU attacks from billions/second to thousands/second. Use bcrypt or Argon2id for even stronger password hashing.

Why It Matters

Timing attacks are a real attack vector that has been used to break cryptographic implementations in production systems. Stripe's webhook signature verification documentation explicitly warns about timing attacks and requires constant-time comparison. Any webhook endpoint that handles payment events, user data changes, or authorization callbacks must use constant-time comparison - the identity-vault implementation provides a correct, reusable reference for this critical security control.

Chapter 38

Google OAuth2 Flow

Gmail, Calendar, and YouTube API Authentication

Project: personal-agents

Overview

Google's OAuth2 flow grants AI agents access to Gmail, Google Calendar, YouTube, and other Google services on behalf of a user without the user sharing their password. The personal-agents project implements the complete OAuth2 authorization code flow: generating the authorization URL, handling the callback with the authorization code, exchanging for access and refresh tokens, storing tokens securely, and automatically refreshing expired access tokens before every API call.

The implementation uses google-auth and google-auth-oauthlib libraries which abstract the OAuth2 protocol details. The key challenge is offline access: the agent needs to make API calls autonomously even when the user is not present to re-authorize. The offline access type and refresh token storage pattern enables this, allowing the agent to operate autonomously for indefinitely long periods after the initial user authorization.

How It Works

- OAuth2 flow begins by generating an authorization URL with requested scopes and redirect URI
- The user visits the URL, authorizes the application, and is redirected with an authorization code
- The code is exchanged for access token (short-lived) and refresh token (long-lived) via token endpoint
- Tokens are stored encrypted to disk (using Fernet from Chapter 36) for persistence across restarts
- google.auth.credentials.Credentials automatically refreshes expired access tokens using the refresh token
- API clients are built with the credentials object which handles token refresh transparently
- Token revocation endpoint allows users to disconnect the agent from their Google account

auth/google_oauth.py - Complete Google OAuth2 flow with auto token refresh

```python
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build

SCOPES = [
    "https://www.googleapis.com/auth/gmail.readonly",
    "https://www.googleapis.com/auth/calendar",
    "https://www.googleapis.com/auth/youtube.upload",
]

def get_credentials(token_file: str = "token.json") -> Credentials:
    creds = None
    if Path(token_file).exists():
        creds = Credentials.from_authorized_user_file(token_file, SCOPES)
    if not creds or not creds.valid:
        if creds and creds.expired and creds.refresh_token:
            creds.refresh(Request())  # auto-refresh
        else:
            flow = InstalledAppFlow.from_client_secrets_file(
                "credentials.json", SCOPES
            )
            creds = flow.run_local_server(port=0)
        with open(token_file, "w") as f:
            f.write(creds.to_json())
    return creds

def get_calendar_service():
    return build("calendar", "v3", credentials=get_credentials())

def get_gmail_service():
    return build("gmail", "v1", credentials=get_credentials())
```

■ Request Minimal Scopes

Requesting all available Google scopes appears suspicious to users during the authorization dialog and to Google's abuse detection. Request only the specific scopes your application actually uses. gmail.readonly is sufficient for reading emails - do not request gmail if you only need calendar access. Minimal scopes also reduce the damage scope of a compromised token.

■■ Encrypt the Token File

The token.json file contains the refresh token, which grants offline access to all authorized Google services indefinitely. Encrypt this file at rest using the Fernet vault from Chapter 36. A compromised token file is equivalent to having the user's Google password for the authorized scopes.

Why It Matters

Google OAuth2 is the gateway to the most widely used productivity suite in the world. Personal AI agents that can read Gmail, manage Calendar events, and upload YouTube videos become genuine force multipliers for their users. The auto-refresh pattern is what enables autonomous operation - without it, the agent would require user re-authorization every hour when the access token expires, making autonomous operation

impossible.

Chapter 39

FastAPI Lifespan Pattern

Startup and Shutdown Hooks for Resource Management

Project: identity-vault

Overview

FastAPI applications often need to initialize resources at startup (database connections, AI model loading, cache warming) and cleanly release them at shutdown (closing connections, flushing write buffers, saving state). The lifespan pattern, introduced in FastAPI 0.93 using Python's contextmanager protocol, provides a clean way to manage application-level resources without relying on deprecated on_event decorators or global variables.

The identity-vault uses the lifespan pattern to initialize the database connection pool, load the encryption key from the secret store, warm the session cache, and verify all dependencies are reachable before accepting traffic. On shutdown, the lifespan context manager closes the database pool, flushes any pending writes, and logs the clean shutdown event. This ensures the application never starts in a partially initialized state and always shuts down gracefully.

How It Works

- The lifespan context manager is a generator function decorated with @asynccontextmanager
- Code before the yield statement runs on startup: db init, key loading, dependency checks
- The yield statement marks where the application runs and serves requests
- Code after the yield runs on shutdown: connection closing, buffer flushing, state saving
- Exceptions during startup propagate and prevent the application from starting
- The lifespan function is passed to FastAPI() constructor as the lifespan parameter
- Application state (db, cipher, cache) is stored in app.state for access from route handlers

main.py - FastAPI lifespan context manager for resource management

```python
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # ■■ STARTUP ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    print("Starting identity-vault...")
    app.state.db = await create_database_pool(
        DATABASE_URL, min_size=2, max_size=10
    )
    app.state.cipher = VaultCipher(
        master_key=os.environ["VAULT_MASTER_KEY"]
    )
    app.state.cache = await init_session_cache()
    # Verify dependencies are reachable
    async with app.state.db.acquire() as conn:
        await conn.execute("SELECT 1")
    print("Identity vault ready")

    yield  # Application runs here

    # ■■ SHUTDOWN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    print("Shutting down identity-vault...")
    await app.state.cache.close()
    await app.state.db.close()
    print("Clean shutdown complete")

app = FastAPI(lifespan=lifespan)

@app.get("/health")
async def health(request: Request):
    await request.app.state.db.fetchone("SELECT 1")
    return {"status": "healthy"}
```

■■ Fail Fast on Startup Dependency Errors

If the database is unreachable or the master key is missing, the application should fail immediately on startup with a clear error message rather than starting and returning 500 errors on the first request. The startup health check in the lifespan catches these issues before the application begins accepting traffic.

■ Store App State in app.state, Not Global Variables

Global variables in FastAPI create thread-safety issues with multiple workers and make testing difficult. Store application-level resources in app.state which is created fresh for each FastAPI instance. Route handlers access it via request.app.state, making test injection straightforward.

Why It Matters

The lifespan pattern is the correct way to manage application resources in modern FastAPI applications. The previous on_event("startup") approach is deprecated and does not compose well with dependency injection testing. The lifespan pattern enables clean resource initialization, health verification before accepting traffic, and guaranteed cleanup on shutdown - the three requirements of production-grade web application resource management.

PART 9

Database & Persistence Patterns

Chapter 40

Prisma + TypeScript - Transaction-Safe Knowledge Pipeline

Type-Safe ORM with ACID Transaction Guarantees

Project: knowledge-vault

Overview

Knowledge extraction pipelines that write multiple related records must do so atomically: if storing a YouTube video's concepts, insights, and summary, either all records are stored successfully or none are - a partial write would leave the database in an inconsistent state where a video has concepts but no summary, or a summary that does not match the stored concepts. The knowledge-vault uses Prisma ORM's transaction API to guarantee atomic multi-record writes with full TypeScript type safety.

Prisma generates a fully-typed client from the schema definition, providing autocompletion for every model field and catching schema violations at compile time rather than runtime. The transaction API executes multiple database operations as a single ACID unit: if any operation fails, all changes are rolled back automatically. This prevents the partial-write corruptions that plague pipelines using raw SQL without explicit transaction management.

How It Works

- schema.prisma defines all models with types, relationships, and constraints enforced by the ORM
- prisma generate creates a TypeScript client with full type inference for every model
- db.$transaction() wraps multiple Prisma operations in an atomic database transaction
- If any operation inside $transaction() throws, all previous operations in the transaction are rolled back
- Nested transactions (savepoints) are supported for complex multi-step pipelines
- Prisma's connection pool manages database connections efficiently across concurrent requests
- Migrations are managed by prisma migrate which tracks schema changes as versioned SQL files

pipeline/processor.ts - Transaction-safe knowledge extraction with Prisma

```
import { PrismaClient } from "@prisma/client"
const db = new PrismaClient()

async function processSource(sourceId: string): Promise<void> {
  const source = await db.source.update({
    where: { id: sourceId },
    data: { status: "processing" },
  })
  const videoData = await fetchYouTubeVideo(source.url)
  const extraction = await extractKnowledge(
    videoData.title, videoData.transcript
  )
  // Atomic: all writes succeed or all roll back
  await db.$transaction(async (tx) => {
    await tx.source.update({
      where: { id: sourceId },
      data: {
        title:   videoData.metadata.title,
        summary: extraction.summary,
        status:  "ready",
      },
    })
    for (const concept of extraction.concepts) {
      await tx.concept.create({
        data: {
          sourceId,
          name:        concept.name,
          explanation: concept.explanation,
          confidence:  concept.confidence,
        },
      })
    }
    for (const insight of extraction.insights) {
      await tx.insight.create({
        data: { sourceId, text: insight.text, importance: insight.importance },
      })
    }
  })
}
```

■■ Use $transaction for All Multi-Record Writes

Any pipeline that writes to more than one table in sequence must use a transaction. Without a transaction, a process crash or network failure between writes leaves the database inconsistent. With $transaction, the database guarantees that all writes complete or none do. This is not optional - it is the correct default for all multi-step writes.

■ **Prisma Type Safety Catches Schema Violations Early**	When you rename a field in schema.prisma, prisma generate regenerates the TypeScript client and TypeScript's compiler flags every reference to the old field name across your entire codebase. This compile-time checking catches schema migration issues that would otherwise surface as runtime errors in production.

Why It Matters

Data consistency is the foundation of trustworthy applications. A knowledge vault with partial writes - concepts without insights, insights without summaries - produces unreliable search results and confusing user experiences. The Prisma transaction pattern guarantees that knowledge-vault data is always consistent: every stored source has all its related records or none of them. The TypeScript type safety layer eliminates an entire class of runtime errors that plague dynamically typed database clients.

Chapter 41

Async SQLite with aiosqlite

Non-Blocking Database Operations in Async Python

Project: personal-agents

Overview

Python's standard sqlite3 module performs blocking I/O operations that freeze the asyncio event loop while waiting for disk reads and writes. In an async application that handles multiple concurrent WebSocket connections and background tasks, a single slow database query can block all other operations. The personal-agents project uses aiosqlite to wrap all database operations in async coroutines that yield control to the event loop during I/O, enabling true concurrent operation.

aiosqlite is a thin async wrapper around sqlite3 that runs database operations in a thread pool, converting blocking calls into awaitable coroutines. The usage pattern mirrors sqlite3 exactly but with async/await keywords, making migration from synchronous code straightforward. Connection management uses async context managers that guarantee connections are properly closed even when exceptions occur.

How It Works

- aiosqlite.connect() opens an async connection that wraps sqlite3 in a thread executor
- All query methods (execute, fetchall, fetchone) are async coroutines awaited with await
- Transactions are managed with async context managers (async with conn) for automatic commit/rollback
- WAL mode (Write-Ahead Logging) enables concurrent reads during write operations for better throughput
- Connection pooling is simulated by maintaining a module-level connection and reusing it across calls
- Prepared statements via parameterized queries prevent SQL injection and improve parse caching
- RETURNING clause retrieves inserted values without a separate SELECT, reducing round trips

db/async_db.py - aiosqlite with WAL mode and upsert operations

```python
import aiosqlite
from pathlib import Path

DB_PATH = Path("data/agents.db")

async def init_db():
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("PRAGMA journal_mode=WAL")
        await db.execute("""
            CREATE TABLE IF NOT EXISTS contacts (
                phone       TEXT PRIMARY KEY,
                name        TEXT,
                preferences TEXT DEFAULT '{}',
                total_calls INTEGER DEFAULT 0,
                last_summary TEXT
            )
        """)
        await db.execute("""
            CREATE TABLE IF NOT EXISTS call_logs (
                id          INTEGER PRIMARY KEY AUTOINCREMENT,
                phone       TEXT NOT NULL,
                summary     TEXT,
                sentiment   TEXT,
                created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
            )
        """)
        await db.commit()

async def upsert_contact(
    phone: str, name: str, preferences: dict
) -> None:
    import json
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            """INSERT INTO contacts (phone, name, preferences)
               VALUES (?, ?, ?)
               ON CONFLICT(phone) DO UPDATE SET
               name=excluded.name, preferences=excluded.preferences""",
            (phone, name, json.dumps(preferences))
        )
        await db.commit()
```

■ WAL Mode Is Essential for Concurrent Access

SQLite's default journal mode locks the entire database file for writes, blocking all readers. WAL (Write-Ahead Logging) mode allows concurrent readers during write operations, dramatically improving throughput for applications with multiple async tasks accessing the database simultaneously.

■■ Always Use Parameterized Queries

Never format values directly into SQL strings: f"SELECT * FROM users WHERE id={user_id}" is a SQL injection vulnerability. Always use parameterized queries with ? placeholders: cursor.execute("SELECT * FROM users WHERE id=?", (user_id,)). This applies regardless of whether the value comes from user input.

Why It Matters

SQLite is underrated as a production database for single-server applications with moderate load. It requires no separate server process, has excellent read performance, and handles thousands of write operations per second in WAL mode. aiosqlite makes SQLite fully compatible with asyncio applications by eliminating the blocking I/O that would otherwise freeze the event loop. For personal agents and similar moderate-scale applications, this combination eliminates PostgreSQL infrastructure complexity without sacrificing correctness or reliability.

Chapter 42

Alembic Migrations

Production Database Schema Versioning

Project: personal-agent

Overview

Database schemas evolve over time: new features require new columns, performance improvements require new indexes, and data model changes require column renames or table restructuring. Alembic is the migration framework for SQLAlchemy that tracks schema changes as versioned Python files, enabling repeatable, reversible migrations that can be applied consistently across development, staging, and production environments.

The personal-agent project uses Alembic to manage the complete schema lifecycle. Each schema change is captured as a migration file with upgrade() and downgrade() functions. Alembic tracks which migrations have been applied in an alembic_version table and applies pending migrations in version order. This ensures that every environment runs the exact same schema version without manual SQL scripts or schema drift.

How It Works

- alembic init creates the alembic/ directory with env.py configuration and versions/ subdirectory
- alembic revision --autogenerate compares SQLAlchemy models to database and generates the migration
- Each revision file has upgrade() with the forward migration and downgrade() to reverse it
- alembic upgrade head applies all pending migrations in order
- alembic downgrade -1 reverts the last applied migration
- The alembic_version table stores the current revision hash to track migration state
- Batch mode handles SQLite's ALTER TABLE limitations using table recreation

alembic/versions/a1b2c3d4_add_memory_store.py - Migration with index creation

```python
"""Add memory_store table and index
Revision: a1b2c3d4e5f6
Revises: 9z8y7x6w5v4u
Create Date: 2025-11-15
"""
from alembic import op
import sqlalchemy as sa

def upgrade():
    op.create_table(
        "memory_store",
        sa.Column("id",         sa.Integer, primary_key=True),
        sa.Column("user_id",    sa.String(64), nullable=False),
        sa.Column("key",        sa.String(255), nullable=False),
        sa.Column("value",      sa.Text),
        sa.Column("created_at", sa.DateTime,
                  server_default=sa.text("CURRENT_TIMESTAMP")),
        sa.Column("updated_at", sa.DateTime,
                  server_default=sa.text("CURRENT_TIMESTAMP"),
                  onupdate=sa.text("CURRENT_TIMESTAMP")),
    )
    op.create_index(
        "ix_memory_store_user_key",
        "memory_store",
        ["user_id", "key"],
        unique=True
    )

def downgrade():
    op.drop_index("ix_memory_store_user_key")
    op.drop_table("memory_store")
```

■■ Always Write down grade()

Skipping downgrade() is tempting when you are confident the migration is correct. Do not. Rollback migrations are the difference between a 5-minute fix and a 2-hour database recovery when something goes wrong in production. Every migration should be fully reversible.

■ Test Migrations on a Staging Database First

Running alembic upgrade head directly on production is high-risk for non-trivial migrations. Apply the migration to a staging database first, run your test suite against the migrated schema, verify no data was corrupted, and only then apply to production. For large tables, plan for migration execution time.

Why It Matters

Schema drift - where different environments run different schema versions due to manual SQL changes - is one of the most common sources of production bugs in database-backed applications. Alembic eliminates

schema drift by making every schema change a tracked, versioned, reproducible artifact. The same migration file that was tested in development runs identically in production. Combined with CI/CD that runs migrations as part of deployment, Alembic makes schema management a solved problem.

Chapter 43

Prisma Multi-Service ORM

RSS, Jobs, Alerts, and Calendar in One Schema

Project: commandcore

Overview

CommandCore's 20-service architecture manages data across many domains: RSS feed articles and read status, job listings and application tracking, price alerts and triggered history, calendar events and reminders, ping results and uptime history, system metrics, and process watchlist. The Prisma multi-service ORM approach defines all of these models in a single schema.prisma file, generating one type-safe client that all services share with proper relation enforcement between related models.

A single unified schema has significant advantages over per-service databases: cross-service queries (finding all alerts that triggered based on RSS articles about a specific topic) are simple Prisma queries rather than complex cross-database joins. The single alembic migration tracks all schema changes atomically. Foreign key constraints between service models enforce data integrity at the database level.

How It Works

- schema.prisma defines all 15+ models for all CommandCore services in a single file
- Relations between models (Article to ReadStatus, Alert to TriggerHistory) use Prisma relation fields
- prisma generate creates a single client with type-safe access to all models and relations
- Each service imports the shared PrismaClient singleton to avoid connection pool exhaustion
- Prisma's findMany with include fetches related records in a single query (no N+1)
- upsert operations handle the common pattern of create-or-update for idempotent operations
- Soft deletes use a deletedAt timestamp rather than hard deletes for audit trail preservation

prisma/schema.prisma - Multi-service schema with relations and indexes

```
// schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model RssFeed {
  id        String    @id @default(cuid())
  url       String    @unique
  name      String
  articles  Article[]
  createdAt DateTime  @default(now())
}

model Article {
  id        String   @id @default(cuid())
  feedId    String
  feed      RssFeed  @relation(fields: [feedId], references: [id])
  title     String
  url       String   @unique
  summary   String?
  sentiment String?  // LONG | SHORT | NEUTRAL
  readAt    DateTime?
  createdAt DateTime @default(now())
  @@index([feedId, createdAt])
}

model PriceAlert {
  id          String    @id @default(cuid())
  ticker      String
  threshold   Float
  direction   String    // ABOVE | BELOW
  triggers    AlertTrigger[]
  isActive    Boolean   @default(true)
  createdAt   DateTime  @default(now())
}
```

■■ One Client Instance, Share It Everywhere

Creating a new PrismaClient() in every file creates a new connection pool, quickly exhausting the database's maximum connection limit. Create a single PrismaClient instance in a shared module and import it everywhere. In Next.js and similar frameworks with hot reload, cache it on globalThis to prevent pool recreation on each reload.

■ **Compound Indexes for Common Query Patterns**	The @@index([feedId, createdAt]) index on Article makes "get recent articles for a feed" queries fast even with millions of articles. Always add indexes for the columns you filter and sort by in your most common queries. Identify these by profiling slow queries in development before they become production problems.

Why It Matters

The single schema approach for multi-service applications trades schema size for developer velocity and data integrity. Every CommandCore service benefits from the same type-safe client, the same migration system, and the same constraint enforcement. Cross-service features (like showing sentiment-analyzed RSS articles alongside price alerts for the same stock) become trivial joins rather than complex inter-service API calls. This architectural choice is the right one for personal and small-team projects where one developer maintains all services.

PART 10

Knowledge Extraction & RAG Patterns

Chapter 44

YouTube Transcript + AI Knowledge Extraction Pipeline

Automated Learning from Video Content

Project: knowledge-vault

Overview

YouTube contains millions of hours of educational content — technical tutorials, university lectures, expert interviews, and conference talks — but that knowledge is locked inside video files that cannot be searched, queried, or cross-referenced. The knowledge-vault YouTube extraction pipeline unlocks this content through a six-stage transformation that is worth understanding end-to-end. The full data flow is: YouTube URL → extract video ID → fetch auto-generated transcript via youtube-transcript-api → concatenate transcript segments into full text → send to AI extraction LLM with a structured output prompt → receive JSON with concepts, insights, quotes, and summary → wrap all database writes in a Prisma $transaction → store source record + concept rows + insight rows atomically, marking status as ready only when all writes succeed. The entire pipeline completes in under 30 seconds for a 60-minute lecture, converting a passive YouTube URL into a queryable knowledge node.

The power becomes clear when you see what gets stored. A 45-minute lecture on neural networks enters as a URL and exits as: a 3-sentence summary suitable for quick recall, 12 extracted concepts each with a plain-English explanation, 8 key insights ranked by importance score, and 4 memorable quotes with timestamps. Every concept and insight is linked back to its source via sourceId, enabling cross-video queries like "what have I learned about backpropagation across all lectures I have ingested?" The Prisma $transaction is the safety mechanism: if the AI extraction succeeds but any database write fails, the source status remains processing and the pipeline will retry — no video is ever silently marked done with partial or missing knowledge objects.

How It Works

- YouTube video ID is extracted from URL and used to fetch title, description, and duration via YouTube API
- youtube-transcript-api fetches the auto-generated or manual transcript in the video language
- Long transcripts are chunked into 3000-token segments with 200-token overlap for context continuity
- Each chunk is processed by the extraction LLM which identifies concepts, insights, and key quotes
- Extracted objects from all chunks are deduplicated by semantic similarity before storage

- Prisma $transaction stores all extracted objects atomically (source + concepts + insights)
- The source status is updated from processing to ready when all extractions are stored

pipeline/youtube_processor.ts - Transcript fetch and atomic knowledge storage

```
import { YoutubeTranscript } from "youtube-transcript"
const db = new PrismaClient()

async function processSource(sourceId: string): Promise<void> {
  const source = await db.source.update({
    where: { id: sourceId }, data: { status: "processing" }
  })
  // Fetch transcript
  const transcript = await YoutubeTranscript.fetchTranscript(
    extractVideoId(source.url)
  )
  const fullText = transcript.map(t => t.text).join(" ")

  // Extract knowledge with AI
  const extraction = await extractKnowledge(
    source.title, fullText, aiProvider
  )

  // Store atomically
  await db.$transaction(async (tx) => {
    await tx.source.update({
      where: { id: sourceId },
      data: { summary: extraction.summary, status: "ready" }
    })
    await tx.concept.createMany({
      data: extraction.concepts.map(c => ({
        sourceId, name: c.name, explanation: c.explanation
      }))
    })
    await tx.insight.createMany({
      data: extraction.insights.map(i => ({
        sourceId, text: i.text, importance: i.importance
      }))
    })
  })
}
```

■ Chunk Transcripts with Overlap

A 90-minute lecture may have 30,000 tokens - far beyond any LLM context window. Chunk the transcript into 3000-token pieces with 200-token overlap at sentence boundaries. The overlap ensures that concepts split across chunk boundaries are fully captured in at least one chunk without being truncated mid-thought.

■ Auto-Generated Transcripts Have Quality Limits

YouTube auto-generated transcripts for technical content often garble technical terms, proper nouns, and code-related vocabulary. Include a transcript quality note in your extraction prompt so the LLM can flag low-confidence extractions. For critical content, consider using Whisper for higher-quality transcription.

■ Common Pitfall: Transcripts Without Chapter Markers Lose Structure

Auto-generated YouTube transcripts are a single continuous text block with no speaker labels, topic transitions, or slide markers. A 90-minute lecture covering 8 distinct topics produces one undifferentiated text blob where the AI must guess which concepts belong together. Before extraction, fetch the video's chapter markers from the YouTube API (available in video.snippet.description or via the chapters endpoint) and use them to split the transcript into topically coherent sections. Extract each section independently for dramatically more relevant concept groupings.

Why It Matters

The average knowledge worker watches 2-5 hours of educational video content per week but retains and acts on only a fraction of what they watch. The YouTube extraction pipeline transforms passive viewing into active knowledge accumulation — every video automatically populates the knowledge vault with searchable, cross-referenceable concepts and insights. Over time, the vault becomes a personal knowledge graph that reflects everything learned through video. Paired with the RAG pipeline in Chapter 45, it enables queries like "what did the lectures I watched last month say about transformer attention mechanisms?" answered by synthesizing across multiple sources — a capability that would take hours to replicate manually.

Chapter 45

Multi-Provider RAG - Gemini + Ollama Abstraction

Retrieval-Augmented Generation with Provider Flexibility

Project: knowledge-vault

Overview

Retrieval-Augmented Generation (RAG) enhances LLM responses by retrieving relevant documents from a knowledge base and including them in the prompt context. The quality and cost of RAG depends on two AI components: the embedding model (which converts text to vectors for similarity search) and the generation model (which synthesizes retrieved content into answers). The knowledge-vault implements a multi-provider RAG system that can use Gemini or Ollama for both components, enabling cost optimization and offline operation.

The RAG pipeline has three stages: embedding (convert the user query to a vector using the embedding model), retrieval (find the most similar stored knowledge chunks using cosine similarity), and generation (send retrieved chunks plus the query to the generation model for synthesis). The provider abstraction allows each stage to use a different provider - for example, Ollama embeddings (free, local) with Gemini generation (high quality, hosted).

How It Works

- The embedding provider converts query and stored documents to high-dimensional vectors
- Vectors are stored in SQLite as BLOB alongside their source text for retrieval
- At query time, the query is embedded and cosine similarity is computed against all stored vectors
- The top-k most similar chunks are retrieved and formatted as context for the generation prompt
- The generation prompt template includes retrieved context and instructs the model to cite sources
- Provider abstraction allows mixing providers: Ollama for local embedding, Gemini for generation
- Response includes source citations with source title and relevance score for traceability

rag/pipeline.py - Multi-provider RAG with embedding, retrieval, and generation stages

```python
class RAGPipeline:
    def __init__(self, embed_provider, gen_provider, db):
        self.embedder = embed_provider
        self.generator = gen_provider
        self.db = db

    async def query(self, question: str, top_k: int = 5) -> RAGResponse:
        # Stage 1: Embed the query
        query_vec = await self.embedder.embed(question)

        # Stage 2: Retrieve similar chunks
        chunks = await self.db.fetchall(
            "SELECT text, source_title, embedding FROM chunks"
        )
        scored = [
            (cosine_sim(query_vec, c["embedding"]), c)
            for c in chunks
        ]
        top_chunks = sorted(scored, reverse=True)[:top_k]

        # Stage 3: Generate with context
        context = "\n\n".join(
            f"[{c['source_title']}]\n{c['text']}"
            for _, c in top_chunks
        )
        prompt = (
            f"Answer based on these sources:\n{context}\n\n"
            f"Question: {question}\n"
            "Cite sources by title in your answer."
        )
        answer = await self.generator.generate(prompt)
        return RAGResponse(
            answer=answer,
            sources=[c["source_title"] for _, c in top_chunks],
            scores=[score for score, _ in top_chunks],
        )
```

■ Pre-Compute and Store Embeddings

Computing embeddings at query time for all stored documents scales O(n) with the number of documents. Pre-compute embeddings when documents are stored and save them in the database. At query time, only the query needs to be embedded - retrieval is just a dot product computation against stored vectors.

■ Ollama for Development, Gemini for Production

Ollama runs embedding and generation models entirely locally - zero cost, zero latency dependency, perfect for development. Switch to Gemini or OpenAI for production where quality matters more than cost. The provider abstraction makes this switch a one-line environment variable change.

Why It Matters

RAG is the architecture that makes AI assistants genuinely useful for personal knowledge management. Without retrieval, the LLM can only answer from its training data, which does not include your personal notes, private documents, or recently learned content. With RAG over a personal knowledge vault, the LLM answers from your specific knowledge - concepts you have learned, insights you have captured, and documents you have stored. This is the pattern behind enterprise search, documentation assistants, and personal AI tutors.

Chapter 46

Article Scraper - Summarize, Categorize, Store

Automated Web Article Processing Pipeline

Project: knowledge-vault

Overview

The knowledge-vault does not limit itself to YouTube videos. Web articles, blog posts, research papers, and news stories all contain valuable knowledge worth capturing. The article processing pipeline uses Playwright for JavaScript-rendered article extraction (since many sites render content dynamically), AI-powered summarization to compress lengthy articles into key points, automatic categorization into the user's topic taxonomy, and transactional storage of the processed article with metadata.

The pipeline handles the messy reality of web content extraction: paywalled articles that require fallback to cached versions, articles with heavy ad markup that obscures the actual content, and sites that block scraping with bot detection. The readability-based content extraction (using Mozilla Readability algorithm) strips navigation, ads, and boilerplate to extract the article body before sending to the AI processing stage.

How It Works

- URL is normalized and checked against the deduplication database before processing begins
- Playwright fetches the full rendered page with JavaScript execution and ad-blocking enabled
- Mozilla Readability algorithm extracts the article body, title, author, and publication date
- The extracted text is sent to the AI summarization model with a structured output prompt
- The AI also assigns up to 3 topic categories from the user's predefined taxonomy
- Extracted metadata (reading time, source domain, authors) is stored alongside the AI summary
- All processing outputs are stored in a single Prisma transaction for atomicity

pipeline/article_processor.py - Playwright + Readability + AI article processing

```python
from playwright.async_api import async_playwright
from readability import Document

async def process_article(url: str) -> ProcessedArticle:
    # Extract with Playwright for JS-rendered content
    async with async_playwright() as pw:
        browser = await pw.chromium.launch(headless=True)
        page = await browser.new_page()
        # Block ads and trackers for cleaner extraction
        await page.route("**/*", lambda r: r.abort()
            if r.request.resource_type in ("image", "media", "font")
            else r.continue_())
        await page.goto(url, wait_until="domcontentloaded")
        html = await page.content()
        await browser.close()

    # Readability extraction
    doc = Document(html)
    content = doc.summary()  # cleaned article body HTML
    title = doc.title()

    # AI processing
    prompt = (
        "Summarize this article and categorize it.\n"
        "Return JSON: {summary, categories: [str], key_points: [str]}\n"
        f"Title: {title}\n"
        f"Content: {content[:4000]}"
    )
    result = json.loads(await ai.generate(prompt))
    return ProcessedArticle(
        url=url, title=title, summary=result["summary"],
        categories=result["categories"],
        key_points=result["key_points"],
    )
```

■ Block Images and Media for Faster Scraping

Images and media files can account for 90% of a page's download size but contribute nothing to text extraction. Blocking these resource types in Playwright reduces page load time from 5-10 seconds to under 1 second for most article pages, while the text content is unaffected.

■ Readability Beats Custom CSS Selectors

Building custom CSS selectors for each website is maintenance-intensive and breaks when sites redesign. Mozilla Readability uses the same algorithm as Firefox Reader Mode - it reliably extracts article content from most news sites, blogs, and documentation pages without site-specific configuration.

Why It Matters

The web is the largest repository of human knowledge ever assembled, but most of it exists in a format that is difficult to search, summarize, or reason over. The article processing pipeline makes the web's content machine-readable and semantically organized, transforming passive reading lists into active knowledge stores. Researchers, analysts, investors, and lifelong learners all benefit from a system that automatically distills and organizes the articles they read into queryable, cross-referenced knowledge assets.

PART 11

Testing, Deployment & DevOps

Chapter 47

Vitest Test Suite - Unit, Integration, and Bot Tests

Comprehensive Testing for TypeScript Services

Project: commandcore

Overview

CommandCore's 20 interconnected services cannot be safely modified without comprehensive tests that verify each service works correctly and that changes to one service do not break others. The Vitest test suite covers three layers: unit tests for pure functions (Kelly criterion calculations, sentiment classification, price normalization), integration tests for service interactions with mocked external APIs, and bot tests that simulate Telegram commands end-to-end through the complete service stack.

Vitest is the modern TypeScript-native testing framework that provides Jest-compatible APIs with native ESM support, TypeScript without transpilation, and 10-20x faster test execution than Jest. The test structure mirrors the source directory, making it easy to find tests for any component. The MockAIProvider from Chapter 2 is the key enabler for deterministic AI-dependent tests.

How It Works

- Vitest discovers test files matching **/*.test.ts or **/*.spec.ts patterns
- Unit tests are pure: no I/O, no network, no database - only function inputs and outputs
- vi.mock() replaces module dependencies with configurable mock implementations
- Integration tests use an in-memory SQLite database to test database-dependent code paths
- Bot tests create a Test Bot instance and simulate command messages without a real Telegram connection
- Snapshot tests verify that complex formatted outputs (briefing messages, reports) maintain expected structure
- Coverage reporting with v8 identifies untested code paths requiring additional test cases

tests/ - Vitest unit, integration, and bot command test examples

```
// tests/signals/kelly.test.ts
import { describe, it, expect } from "vitest"
import { kelly_fraction, size_position } from "@/signals/kelly"

describe("Kelly Criterion", () => {
  it("returns 0 for no edge", () => {
    expect(kelly_fraction(0.45, 2.0)).toBe(0)  // negative edge
  })
  it("computes correct fraction for 60% win at 2x", () => {
    // k = (0.6 * 2 - 0.4) / 2 = 0.8/2 = 0.4
    expect(kelly_fraction(0.60, 2.0)).toBeCloseTo(0.4)
  })
  it("sizes position with half-Kelly and 10% cap", () => {
    const size = size_position(10000, 0.60, 2.0, 0.10)
    // half-Kelly = 0.2, capped at 0.10, so 10000 * 0.1 = 1000
    expect(size).toBe(1000)
  })
})

// tests/bot/brief.test.ts
import { vi, describe, it, expect } from "vitest"
import { createBot } from "@/bot"
import { MockAIProvider } from "@/ai"

vi.mock("@/ai/provider", () => ({ getAIProvider: () => new MockAIProvider() }))

describe("Morning Briefing Command", () => {
  it("responds to /brief with formatted message", async () => {
    const responses: string[] = []
    const bot = createBot({ TELEGRAM_TOKEN: "test" })
    // Simulate /brief command
    await bot.handleUpdate({
      message: { text: "/brief", chat: { id: 123 }, from: { id: 1 } }
    })
    // Verify response was sent
    expect(responses[0]).toContain("Morning Brief")
  })
})
```

■ Test Pure Functions First

Pure functions (Kelly criterion, EV calculation, score normalization) are the easiest and most valuable tests to write. They have no dependencies, are always deterministic, and encode the core business logic. Start with 100% coverage of pure functions before tackling the harder-to-test integration layers.

■ MockAIProvider Enables Deterministic AI Tests

Tests that make real AI API calls are slow, expensive, and non-deterministic. MockAIProvider returns configured fixture responses instantly. This enables testing the AI-dependent parts of your application (intent classification, content generation, data extraction) without any API calls, making tests 100x faster and completely reproducible.

Why It Matters

A multi-service system without tests is a house of cards - each change risks breaking something elsewhere, and the developer never knows what is safe to modify. The three-layer test strategy (unit, integration, bot) catches different categories of bugs at appropriate levels of cost: fast unit tests catch logic errors, integration tests catch service interaction bugs, and bot tests catch command-handling regressions. This coverage is what enables confident refactoring and feature addition without fear.

Chapter 48

Docker + Docker Compose - Production Containerization

Reproducible Deployments with Service Orchestration

Project: identity-vault

Overview

Running an application directly on a host server creates invisible dependencies on the system Python version, installed libraries, environment variables, and system configuration that make deployments fragile and non-reproducible. Docker packages everything the application needs into a self-contained image that runs identically on any host with Docker installed. Docker Compose orchestrates multi-service applications - FastAPI server, PostgreSQL database, Redis cache, and Nginx reverse proxy - with a single command.

The identity-vault uses a multi-stage Docker build to produce lean production images: the builder stage installs build tools and compiles dependencies, the final stage copies only the compiled dependencies and application code, resulting in images 3-5x smaller than single-stage builds. Docker Compose handles service dependencies, environment variable injection, volume mounting for persistent data, and network configuration between services.

How It Works

- Multi-stage Dockerfile: builder stage (full Python with dev tools) and final stage (slim runtime only)
- Final stage copies only the virtual environment from builder, not the build tools
- .dockerignore prevents test files, local configs, and __pycache__ from being included in the image
- docker-compose.yml defines all services, their dependencies, environment variables, and volumes
- depends_on with healthcheck condition ensures the database is ready before the app starts
- Named volumes persist PostgreSQL data across container restarts and image rebuilds
- Network isolation keeps services on a private bridge network with only Nginx exposed publicly

Dockerfile and docker-compose.yml - Multi-stage build with service orchestration

```
# Dockerfile - Multi-stage build
FROM python:3.12-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

---

# docker-compose.yml
version: "3.9"
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: identity_vault
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASS}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "${DB_USER}"]
      interval: 5s
      retries: 5

  api:
    build: .
    environment:
      DATABASE_URL: postgresql://${DB_USER}:${DB_PASS}@db:5432/identity_vault
      VAULT_MASTER_KEY: ${VAULT_MASTER_KEY}
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "8000:8000"

volumes:
  pgdata:
```

■ Multi-Stage Builds Reduce Image Size Dramatically

A single-stage Python image with build tools and development dependencies can reach 2-3GB. Multi-stage builds that copy only the installed packages (not the build tools that created them) typically produce images under 300MB. Smaller images mean faster pulls, less attack surface, and lower storage costs.

■■ Use healthcheck with depends_on

depends_on without a condition only waits for the container to start, not for the service inside to be ready. A PostgreSQL container starts in seconds but takes 2-3 more seconds for the database to accept connections. Without the healthcheck condition, your API will fail to connect on startup. Always pair depends_on with service_healthy.

Why It Matters

Docker is the universal deployment substrate that makes "works on my machine" a solved problem. Every developer runs the same container, CI/CD deploys the same container, and production runs the same container - eliminating the configuration drift that plagues traditional deployment approaches. The Docker Compose orchestration pattern shown here is how teams deploy multi-service applications from the smallest personal project to production systems handling millions of requests.

Chapter 49

IP-Based Rate Limiting with Supabase

Protecting AI APIs from Abuse and Cost Overruns

Project: agent-base

Overview

An unprotected AI API endpoint that makes $0.01 LLM API calls can cost hundreds of dollars in minutes if discovered by an API scraper or abused by a single user. The agent-base project implements IP-based rate limiting backed by Supabase (PostgreSQL-as-a-service) to track usage per IP address with daily reset logic. When an IP exceeds the daily limit, all subsequent requests receive a 429 response without any LLM call being made.

The rate limiting implementation uses a simple two-column table in Supabase (ip_address, usage_count with daily reset). The getUsageCount() and incrementUsage() functions are called at the beginning of every API request before any AI processing begins. Supabase's row-level security (RLS) policies prevent clients from directly manipulating their own usage counts through the Supabase client SDK.

How It Works

- Each API request extracts the client IP from x-forwarded-for or the direct connection address
- getUsageCount() queries Supabase for the current day's usage count for this IP
- If count >= limit (default 10 daily), return 429 immediately before any AI processing
- incrementUsage() atomically increments the count and creates the row if it does not exist
- Daily reset is implemented by including the date in the table key or a date column filter
- Supabase RLS policies prevent unauthenticated clients from bypassing limits via direct API access
- Usage counts are stored with the date to support analytics on daily traffic patterns

lib/rate-limit.ts - IP-based daily rate limiting with Supabase RPC

```
import { createClient } from "@supabase/supabase-js"

const supabase = createClient(SUPABASE_URL, SUPABASE_SERVICE_KEY)
const DAILY_LIMIT = 10

async function getUsageCount(ip: string): Promise<number> {
  const today = new Date().toISOString().split("T")[0]
  const { data } = await supabase
    .from("usage_logs")
    .select("count")
    .eq("ip", ip)
    .eq("date", today)
    .single()
  return data?.count ?? 0
}

async function incrementUsage(ip: string): Promise<void> {
  const today = new Date().toISOString().split("T")[0]
  await supabase.rpc("increment_usage", { p_ip: ip, p_date: today })
}

// SQL function for atomic increment (in Supabase dashboard)
// CREATE OR REPLACE FUNCTION increment_usage(p_ip text, p_date date)
// RETURNS void AS $$
// BEGIN
//   INSERT INTO usage_logs (ip, date, count) VALUES (p_ip, p_date, 1)
//   ON CONFLICT (ip, date) DO UPDATE
//   SET count = usage_logs.count + 1;
// END;
// $$ LANGUAGE plpgsql;
```

■■ Use a Database RPC for Atomic Increment

Fetching the count, adding 1, and updating in separate operations creates a race condition where two concurrent requests can both read count=9, both pass the limit check, and both proceed. A database-level atomic increment (INSERT ... ON CONFLICT DO UPDATE SET count = count + 1) eliminates this race condition completely.

■ Vary Limits by Endpoint Cost

Not all endpoints cost the same. A simple chat completion might cost $0.001 while an embedding + RAG + generation pipeline costs $0.05. Apply tighter limits to expensive endpoints and more generous limits to cheap ones. Track cost per endpoint to set limits that balance user experience against budget.

Why It Matters

Rate limiting is the difference between a public API demo and a surprise $5,000 cloud bill. Every public-facing API endpoint that makes paid LLM calls must have rate limiting. The IP-based approach provides per-user limits without requiring authentication, making it appropriate for anonymous demo applications while still protecting against automated abuse. More sophisticated applications should combine IP rate limiting with

authenticated user limits for layered protection.

Chapter 50

Stripe Webhook + Checkout - Monetization Pattern

Subscription Billing and Webhook-Driven Provisioning

Project: agent-base

Overview

The agent-base project implements the complete Stripe subscription lifecycle as a technical pattern study. The core engineering insight is asymmetry: the checkout flow is synchronous (user clicks, sees a Stripe-hosted page, completes payment in their browser), but provisioning must be asynchronous, event-driven, and critically — idempotent. Idempotent means that processing the same webhook event twice produces exactly the same outcome as processing it once. This matters because Stripe delivers webhooks with at-least-once semantics: if your server returns a non-200 response during processing, Stripe will retry the same event multiple times over the next few hours. Without idempotency checks, a single network timeout during database write could cause double-provisioning, duplicate subscription records, or corrupted subscription state. The fix is simple: store the Stripe event ID on first successful processing and check for it before executing any state changes.

Stripe signature verification (stripe.webhooks.constructEvent) defends against two attack vectors simultaneously. First, it verifies the event came from Stripe and was not tampered with — any modification to the request body invalidates the HMAC-SHA256 signature. Second, the Stripe-Signature header embeds a timestamp, and Stripe rejects verification if the timestamp is more than 5 minutes old, preventing replay attacks: an attacker who intercepts a valid checkout.session.completed webhook cannot re-send it 24 hours later to re-provision a cancelled account. Understanding the subscription state machine is equally important for complete implementations: subscriptions move through created → active → past_due (payment failed, grace period begins) → canceled (grace period expired, access must be revoked). Each transition fires a specific webhook event — checkout.session.completed, customer.subscription.updated, invoice.payment_failed, and customer.subscription.deleted — and your access control must gate features on the current subscription_status column updated by each event, not a simple boolean flag.

How It Works

- A checkout session is created server-side with price ID, success URL, cancel URL, and userId in metadata
- The user is redirected to the Stripe-hosted checkout page — Stripe handles PCI-compliant card processing, not your server
- Stripe calls your webhook URL with checkout.session.completed when payment succeeds — this is the provisioning trigger
- The webhook handler reads the raw request body as text first, then calls constructEvent() to verify the signature
- Signature verification authenticates the event and validates the timestamp to prevent replay attacks
- Store the Stripe event ID in a processed_events table before executing any state changes to ensure idempotency

- On successful payment, update the user record with subscription_status: active and stripeCustomerId
- Handle customer.subscription.deleted to revoke access when subscriptions cancel or payment fails irrecoverably

app/api/ - Checkout session creation and webhook-driven provisioning

```
// app/api/checkout/route.ts - Create checkout session
import Stripe from "stripe"
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

export async function POST(req: NextRequest) {
  const { userId } = await req.json()
  const session = await stripe.checkout.sessions.create({
    mode: "subscription",
    payment_method_types: ["card"],
    line_items: [{ price: PRICE_ID, quantity: 1 }],
    success_url: `${BASE_URL}/success?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url:  `${BASE_URL}/pricing`,
    client_reference_id: userId,
    metadata: { userId },
  })
  return NextResponse.json({ url: session.url })
}

// app/api/webhook/route.ts - Handle Stripe events
export async function POST(req: NextRequest) {
  const body = await req.text()
  const sig  = req.headers.get("stripe-signature")!
  let event: Stripe.Event
  try {
    event = stripe.webhooks.constructEvent(body, sig, WEBHOOK_SECRET)
  } catch {
    return new Response("Invalid signature", { status: 400 })
  }
  if (event.type === "checkout.session.completed") {
    const session = event.data.object as Stripe.CheckoutSession
    await db.user.update({
      where: { id: session.metadata!.userId },
      data: {
        stripeCustomerId: session.customer as string,
        subscriptionStatus: "active",
      }
    })
  }
  return new Response("OK")
}
```

■■ Verify Webhook Signatures Before Processing

stripe.webhooks.constructEvent() verifies the Stripe-Signature header using your webhook secret. Never skip this verification - without it, anyone who knows your webhook URL can send fake payment success events and provision free access. The verification takes 1ms and prevents a critical security vulnerability.

■ Make Webhook Handlers Idempotent

Stripe may deliver webhook events more than once if your endpoint returns a non-200 response. Make your handler idempotent: use the Stripe event ID as a deduplication key, check if it has been processed before, and skip re-processing if it has. This prevents double-provisioning from duplicate webhook deliveries.

■■ Common Pitfall: Reading Parsed Body Breaks Signature Verification

stripe.webhooks.constructEvent() requires the raw, unparsed request body bytes — not the JSON-parsed object. Most web frameworks parse the body into an object before your handler sees it, changing the byte representation and invalidating the signature. In Next.js, use req.text() not req.json(). In Express, configure express.raw() middleware specifically for the webhook route. In FastAPI, use Request.body() rather than a parsed Pydantic model parameter. This is the most common cause of 'Invalid signature' errors from otherwise correct Stripe webhook implementations.

Why It Matters

Stripe with Webhooks is the industry-standard payment integration pattern for SaaS applications, and getting it right requires understanding three concepts working together: idempotency (process each event exactly once even when Stripe retries), signature verification (authenticate events to prevent forgery and replay attacks), and the subscription state machine (track all five subscription lifecycle events, not just the happy path). Mastering this pattern means you can add reliable subscription billing to any application — the checkout → active → past_due → canceled lifecycle applies to every SaaS product, and handling it correctly from day one saves weeks of debugging billing edge cases in production when real user subscriptions start failing.